

컴퓨터 비전 프로젝트

# YOLOv8 기반 제조 데이터 객체 탐지 실습

사전학습 모델 이해 → stamp 데이터 학습·평가 → 모델·조건별 성능 실험

|       |                                                                                                 |
|-------|-------------------------------------------------------------------------------------------------|
| 작성자   | hwang-ye-song                                                                                   |
| 레포지토리 | <a href="https://github.com/hwang-ye-song/CV_YOLO">https://github.com/hwang-ye-song/CV_YOLO</a> |
| 작성일   | 2026-09-18                                                                                      |
| 환경    | Windows 11 · Python 3.11 · PyTorch 2.11 (CUDA 12.8) ·<br>Ultralytics 8.4 · RTX 5060 Laptop      |

## 목차

- 실습 노트북 — yolov8\_manufacturing\_practice.ipynb

# YOLOv8 기반 제조 데이터 객체 탐지 실습 [프로젝트]

| 항목       | 내용                                                                                              |
|----------|-------------------------------------------------------------------------------------------------|
| 목표       | ① 사전학습 YOLOv8의 추론 결과 구조를 이해한다 ② 제조 현장 데이터(stamp: 신발 + 도장)로 학습·평가한다 ③ 모델·데이터 조건을 바꿔 성능 범위를 실험한다  |
| 진행<br>방식 | 과제 기본 코드를 순서대로 따라가며 <b>각 코드 아래에 의미를 정리</b> 하고, 파트가 끝날 때마다 🧪 <b>개인 실험</b> 을 수행                   |
| 환경       | Windows 11 · Python 3.11 · PyTorch 2.11 (CUDA 12.8) · Ultralytics 8.4 · RTX 5060 Laptop 8GB     |
| 레포       | <a href="https://github.com/hwang-ye-song/CV_YOLO">https://github.com/hwang-ye-song/CV_YOLO</a> |

## 목차

- Part 1. YOLOv8 확인하기 (버스 이미지) → 🧪 개인 실험 1
- Part 2. YOLOv8 학습 실습 (stamp 데이터셋) → 🧪 개인 실험 2
- 목표 기준 평가 · 개인 회고 (KPT / AAR)

## Part 1. YOLOv8 확인하기

```
In [1]: !pip install ultralytics
```

Collecting ultralytics

Using cached ultralytics-8.4.154-py3-none-any.whl.metadata (46 kB)

Collecting cloudpickle>=3.1.1 (from ultralytics)

Using cached cloudpickle-3.1.2-py3-none-any.whl.metadata (7.1 kB)

Requirement already satisfied: filelock>=3.16.1 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from ultralytics) (3.29.0)

Requirement already satisfied: numpy>=1.23.0 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from ultralytics) (2.4.6)

Requirement already satisfied: matplotlib>=3.3.0 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from ultralytics) (3.11.0)

Requirement already satisfied: opencv-python!=4.13.0.90,>=4.7.0 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from ultralytics) (5.0.0.93)

Requirement already satisfied: pillow>=7.1.2 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from ultralytics) (12.2.0)

Requirement already satisfied: pyyaml>=5.3.1 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from ultralytics) (6.0.3)

... (37줄 생략) ...

Requirement already satisfied: mpmath<1.4,>=1.1.0 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)

Requirement already satisfied: MarkupSafe>=2.0 in c:\users\loq5060\appdata\local\programs\python\python311\lib\site-packages (from jinja2->torch>=1.8.0->ultralytics) (3.0.3)

Using cached ultralytics-8.4.154-py3-none-any.whl (1.4 MB)

Using cached cloudpickle-3.1.2-py3-none-any.whl (22 kB)

Using cached nvidia\_ml\_py-13.610.43-py3-none-any.whl (53 kB)

Using cached polars-1.44.2-py3-none-any.whl (865 kB)

Using cached polars\_runtime\_32-1.44.2-cp310-abi3-win\_amd64.whl (51.3 MB)

Using cached psutil-7.2.2-cp37-abi3-win\_amd64.whl (137 kB)

Using cached ultralytics\_platform-0.1.46-py3-none-any.whl (78 kB)

Using cached ultralytics\_thop-2.1.6-py3-none-any.whl (30 kB)

Installing collected packages: nvidia-ml-py, psutil, polars-runtime-32, cloudpickle, polars, ultralytics-thop, ultralytics-platform, ultralytics

Successfully installed cloudpickle-3.1.2 nvidia-ml-py-13.610.43 polars-1.44.2 polars-runtime-32-1.44.2 psutil-7.2.2 ultralytics-8.4.154 ultralytics-platform-0.1.46 ultralytics-thop-2.1.6

 **설명** — YOLOv8을 만든 Ultralytics 패키지를 설치한다. 모델 정의·가중치 다운로드·학습·추론·평가 기능이 모두 들어 있다. (이 PC에서는 프로젝트 가상환경에 미리 설치돼 있어서 Requirement already satisfied (이미 설치됨)로 끝난다.)

## Yolo 이해

In [2]: `from ultralytics import YOLO`

 **설명** — `YOLO` 클래스 하나로 탐지·분류·분할 등 모든 작업을 다룬다. 어떤 작업인지는 불러오는 가중치 파일이 정한다.

In [3]: `model = YOLO('yolov8n.pt')`

 **설명** — COCO 데이터셋(80개 클래스)으로 사전학습된 **YOLOv8 nano** 모델을 불러온다. 파일이 없으면 자동으로 받는다. `n` (nano) < `s` < `m` < `l` < `x` 순서로 커지고, 클수록 정확하지만 느리다.

```
In [4]: example_imgs = ['https://ultralytics.com/images/bus.jpg']
```

```
In [5]: from PIL import Image
import requests
from io import BytesIO
```

```
In [6]: response = requests.get(example_imgs[0])
img = Image.open(BytesIO(response.content))
```

```
In [7]: img.thumbnail((400,400))
img
```

Out[7]:



 **설명** — URL의 이미지를 메모리로 받아 PIL 이미지로 연다. `thumbnail` 은 원본 비율을 유지하면서 최대 400×400 안에 들어가게 줄인다. 화면 표시용일 뿐이고, 모델에는 원본 URL을 그대로 넘긴다.

```
In [8]: results = model.predict(example_imgs[0], save=True, conf=0.25)
```

```
Downloading https://ultralytics.com/images/bus.jpg to 'bus.jpg': 100% _____
134.2KB 1.1MB/s 0.1s
image 1/1 C:\Users\LQ5060\Desktop\CV-YOLO\notebooks\bus.jpg: 640x480 4 persons, 1
bus, 1 stop sign, 17.7ms
Speed: 35.6ms preprocess, 17.7ms inference, 17.0ms postprocess per image at shape
(1, 3, 640, 480)
Results saved to C:\Users\LQ5060\Desktop\CV-YOLO\notebooks\runs\detect\predict
```

 **설명** — 추론을 실행한다.

- `save=True` : 박스를 그린 결과 이미지를 `runs/detect/predict*/` 에 저장
- `conf=0.25` : 신뢰도 0.25 미만인 박스는 버림
- 출력 로그의 `640x480` 은 모델 입력 크기다. 긴 변을 640에 맞추고 비율을 유지한다.

```
In [9]: output_img = Image.open('./runs/detect/predict/bus.jpg')
output_img.thumbnail((400,400))
output_img
```

Out[9]:



 **설명** — `save=True` 로 저장된 결과 이미지를 열어 확인한다. 사람 4명, 버스, 정지 표지판에 박스와 이름·신뢰도가 그려져 있다. 같은 셀을 여러 번 실행하면 저장 폴더가 `predict2`, `predict3` ...으로 늘어나므로, 경로를 고정하지 않으려면 `results[0].save_dir` 을 쓰면 된다.

```
In [10]: results
```

Out[10]: [ultralytics.engine.results.Results object with attributes:

```
boxes: ultralytics.engine.results.Boxes object
depth: None
keypoints: None
masks: None
... (42줄 생략) ...
      [103, 93, 99]],
      [[123, 122, 126],
       [145, 144, 148],
       [176, 175, 179],
       ...,
       [ 95, 85, 91],
       [ 96, 86, 92],
       [ 98, 88, 94]]], shape=(1080, 810, 3), dtype=uint8)
orig_shape: (1080, 810)
path: 'C:\\Users\\LQ5060\\Desktop\\CV-YOLO\\notebooks\\bus.jpg'
probs: None
save_dir: 'C:\\Users\\LQ5060\\Desktop\\CV-YOLO\\notebooks\\runs\\detect\\predict'
semantic_mask: None
speed: {'preprocess': 35.64029998960905, 'inference': 17.722400021739304, 'post process': 16.97470000362955}]
```

In [11]: `type(results)`

Out[11]: `list`

In [12]: `type(results[0])`

Out[12]: `ultralytics.engine.results.Results`

 **설명** — `predict()` 는 입력 이미지 한 장마다 `Results` 객체 하나를 담은 리스트를 돌려준다. 이미지가 한 장이라 `results[0]` 만 쓴다. `Results` 안에는 `boxes` (탐지), `masks` (분할), `keypoints` (자세), `probs` (분류), `names` (클래스 이름표)가 있고, 탐지 모델은 `boxes` 만 채워진다.

In [13]: `results[0].boxes`

Out[13]: ultralytics.engine.results.Boxes object with attributes:

```
cls: tensor([ 5.,  0.,  0.,  0.,  0., 11.], device='cuda:0')
conf: tensor([0.8732, 0.8657, 0.8528, 0.8253, 0.2608, 0.2552], device='cuda:0')
data: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02, 8.7323e-01, 5.000
0e+00],
            [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02, 8.6572e-01, 0.0000e+0
0],
            [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02, 8.5275e-01, 0.0000e+0
0],
            [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02, 8.2528e-01, 0.0000e+0
0],
            [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02, 2.6076e-01, 0.0000e+0
0],
            [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02, 2.5521e-01, 1.1000e+0
1]], device='cuda:0')
id: None
is_track: False
orig_shape: (1080, 810)
shape: torch.Size([6, 6])
xywh: tensor([[413.9292, 494.0699, 782.1526, 525.5790],
... (8줄 생략) ...
            [0.3497, 0.5849, 0.1524, 0.4183],
            [0.0389, 0.6592, 0.0778, 0.2990],
            [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
xyxy: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02],
            [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02],
            [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02],
            [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02],
            [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02],
            [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02]], device='cuda:0')
xyxyn: tensor([[2.8214e-02, 2.1415e-01, 9.9383e-01, 7.0080e-01],
            [5.9933e-02, 3.6903e-01, 3.0289e-01, 8.3584e-01],
            [8.2651e-01, 3.6314e-01, 9.9965e-01, 8.1207e-01],
            [2.7347e-01, 3.7574e-01, 4.2589e-01, 7.9403e-01],
            [0.0000e+00, 5.0974e-01, 7.7826e-02, 8.0874e-01],
            [7.1355e-05, 2.3561e-01, 4.0194e-02, 3.0081e-01]], device='cuda:0')
```

 설명 — Boxes 객체의 주요 속성

| 속성            | 내용                                       |
|---------------|------------------------------------------|
| cls           | 클래스 번호 (5=bus, 0=person, 11=stop sign)   |
| conf          | 신뢰도                                      |
| xyxy / xywh   | 픽셀 좌표 (좌상단·우하단 / 중심·크기)                  |
| xyxyn / xywhn | 위 좌표를 이미지 크기로 나눈 0~1 값                   |
| data          | [x1, y1, x2, y2, conf, cls] 를 한 행에 모은 텐서 |

```
In [14]: url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytic
s/cfg/datasets/coco.yaml"
```

```
In [15]: import yaml
import requests

url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/cfg/datasets/coco.yaml"

response = requests.get(url)
data = yaml.safe_load(response.text)

print(data)
```

... (1줄 생략) ...

 **설명** — COCO 데이터셋 설정 파일(yaml)을 읽는다. `names` 에 클래스 번호 → 이름 표가 있어서 `cls` 숫자를 사람이 읽을 수 있는 이름으로 바꿀 때 쓴다. (과제 원본의 `import requests=` 는 문법 오류라서 `import requests` 로 고쳤다.)

```
In [16]: results[0].boxes.conf
```

```
Out[16]: tensor([0.8732, 0.8657, 0.8528, 0.8253, 0.2608, 0.2552], device='cuda:0')
```

 **설명** — 6개 박스의 신뢰도. 버스·사람 3명은 0.8 이상으로 확실하고, 가장자리에 잘린 사람과 정지 표지판은 0.26 정도로 애매하다.

```
In [17]: results[0].boxes
```

Out[17]: ultralytics.engine.results.Boxes object with attributes:

```
cls: tensor([ 5.,  0.,  0.,  0.,  0., 11.], device='cuda:0')
conf: tensor([0.8732, 0.8657, 0.8528, 0.8253, 0.2608, 0.2552], device='cuda:0')
data: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02, 8.7323e-01, 5.0000e+00],
              [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02, 8.6572e-01, 0.0000e+00],
              [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02, 8.5275e-01, 0.0000e+00],
              [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02, 8.2528e-01, 0.0000e+00],
              [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02, 2.6076e-01, 0.0000e+00],
              [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02, 2.5521e-01, 1.1000e+00]], device='cuda:0')
id: None
is_track: False
orig_shape: (1080, 810)
shape: torch.Size([6, 6])
xywh: tensor([[413.9292, 494.0699, 782.1526, 525.5790],
              ... (8줄 생략) ...
              [0.3497, 0.5849, 0.1524, 0.4183],
              [0.0389, 0.6592, 0.0778, 0.2990],
              [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
xyxy: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02],
              [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02],
              [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02],
              [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02],
              [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02],
              [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02]], device='cuda:0')
xyxyn: tensor([[2.8214e-02, 2.1415e-01, 9.9383e-01, 7.0080e-01],
               [5.9933e-02, 3.6903e-01, 3.0289e-01, 8.3584e-01],
               [8.2651e-01, 3.6314e-01, 9.9965e-01, 8.1207e-01],
               [2.7347e-01, 3.7574e-01, 4.2589e-01, 7.9403e-01],
               [0.0000e+00, 5.0974e-01, 7.7826e-02, 8.0874e-01],
               [7.1355e-05, 2.3561e-01, 4.0194e-02, 3.0081e-01]], device='cuda:0')
```

In [18]: `img.size`

Out[18]: (300, 400)

 **설명** — `img` 는 위에서 `thumbnail` 로 줄였기 때문에 (300, 400) 이다. 박스 좌표는 원본(810×1080) 기준이므로, 정규화를 직접 계산하려면 원본 크기가 필요하다 → 아래에서 원본을 다시 받는다. PIL의 `size` 는 (너비, 높이) 순서라는 점도 주의.

In [19]: `results[0].boxes.xywh`

```
Out[19]: tensor([[413.9292, 494.0699, 782.1526, 525.5790],
                  [146.9449, 650.6298, 196.7977, 504.1556],
                  [739.5948, 634.6099, 140.2510, 484.8480],
                  [283.2428, 631.6757, 123.4565, 451.7523],
                  [ 31.5197, 711.9839,  63.0394, 322.9200],
                  [ 16.3075, 289.6674,  32.4993,  70.4186]], device='cuda:0')
```

In [20]: `results[0].boxes.xywhn`

```
Out[20]: tensor([[0.5110, 0.4575, 0.9656, 0.4866],
                 [0.1814, 0.6024, 0.2430, 0.4668],
                 [0.9131, 0.5876, 0.1731, 0.4489],
                 [0.3497, 0.5849, 0.1524, 0.4183],
                 [0.0389, 0.6592, 0.0778, 0.2990],
                 [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
```

```
In [21]: import cv2
import requests
import numpy as np
from PIL import Image
from io import BytesIO

img_url = "https://ultralytics.com/images/bus.jpg"
response = requests.get(img_url)
img = Image.open(BytesIO(response.content)).convert("RGB")
img_np = np.array(img)
height, width = img_np.shape[:2]
```

```
In [22]: height, width
```

```
Out[22]: (1080, 810)
```

 **설명** — NumPy 배열의 `shape` 는 (높이, 너비, 채널) 순서다. 원본은 1080×810.

```
In [23]: bbox1 = results[0].boxes.xywh[0]
bbox1
```

```
Out[23]: tensor([413.9292, 494.0699, 782.1526, 525.5790], device='cuda:0')
```

```
In [24]: bbox1[0]/width, bbox1[1]/height, bbox1[2]/width, bbox1[3]/height
```

```
Out[24]: (tensor(0.5110, device='cuda:0'),
          tensor(0.4575, device='cuda:0'),
          tensor(0.9656, device='cuda:0'),
          tensor(0.4866, device='cuda:0'))
```

```
In [25]: results[0].boxes.xywhn
```

```
Out[25]: tensor([[0.5110, 0.4575, 0.9656, 0.4866],
                 [0.1814, 0.6024, 0.2430, 0.4668],
                 [0.9131, 0.5876, 0.1731, 0.4489],
                 [0.3497, 0.5849, 0.1524, 0.4183],
                 [0.0389, 0.6592, 0.0778, 0.2990],
                 [0.0201, 0.2682, 0.0401, 0.0652]], device='cuda:0')
```

 **설명** — 첫 번째 박스(버스)의 `xywh` 를 원본 너비·높이로 직접 나눈 값이 `xywhn[0]` 과 정확히 같다. → **n 이 붙은 좌표 = 원본 크기로 나눈 비율**이다. 해상도와 무관해서 YOLO 학습 라벨(txt)도 이 `xywhn` 형식을 쓴다.

```
In [26]: bbox2 = results[0].boxes.xyxy[0]
bbox2
```

```
Out[26]: tensor([ 22.8529, 231.2805, 805.0055, 756.8594], device='cuda:0')
```

```
In [27]: bbox2[0]/width, bbox2[1]/height, bbox2[2]/width, bbox2[3]/height
```

```
Out[27]: (tensor(0.0282, device='cuda:0'),
          tensor(0.2141, device='cuda:0'),
          tensor(0.9938, device='cuda:0'),
          tensor(0.7008, device='cuda:0'))
```

```
In [28]: results[0].boxes.xyxy[0]
```

```
Out[28]: tensor([0.0282, 0.2141, 0.9938, 0.7008], device='cuda:0')
```

 **설명** — `xyxy` 도 같은 방식으로 정규화하면 `xyxyn` 과 일치한다. `xyxy` 는 OpenCV 로 사각형을 그릴 때 그대로 쓸 수 있어서 시각화에 편하다.

## 시각화

```
In [29]: url = "https://ultralytics.com/images/bus.jpg"
         response = requests.get(url)
         img = np.array(Image.open(BytesIO(response.content)).convert("RGB"))
         h, w = img.shape[:2]
         img_copy = img.copy()
```

```
In [30]: xyxy_coordinate = results[0].boxes.xyxy.cpu().numpy()
```

```
In [31]: xyxy_coordinate
```

```
Out[31]: array([[ 22.853,  231.28,  805.01,  756.86],
                 [ 48.546,  398.55,  245.34,  902.71],
                 [ 669.47,  392.19,  809.72,  877.03],
                 [ 221.51,  405.8,  344.97,  857.55],
                 [    0,  550.52,  63.039,  873.44],
                 [ 0.057797,  254.46,  32.557,  324.88]], dtype=float32)
```

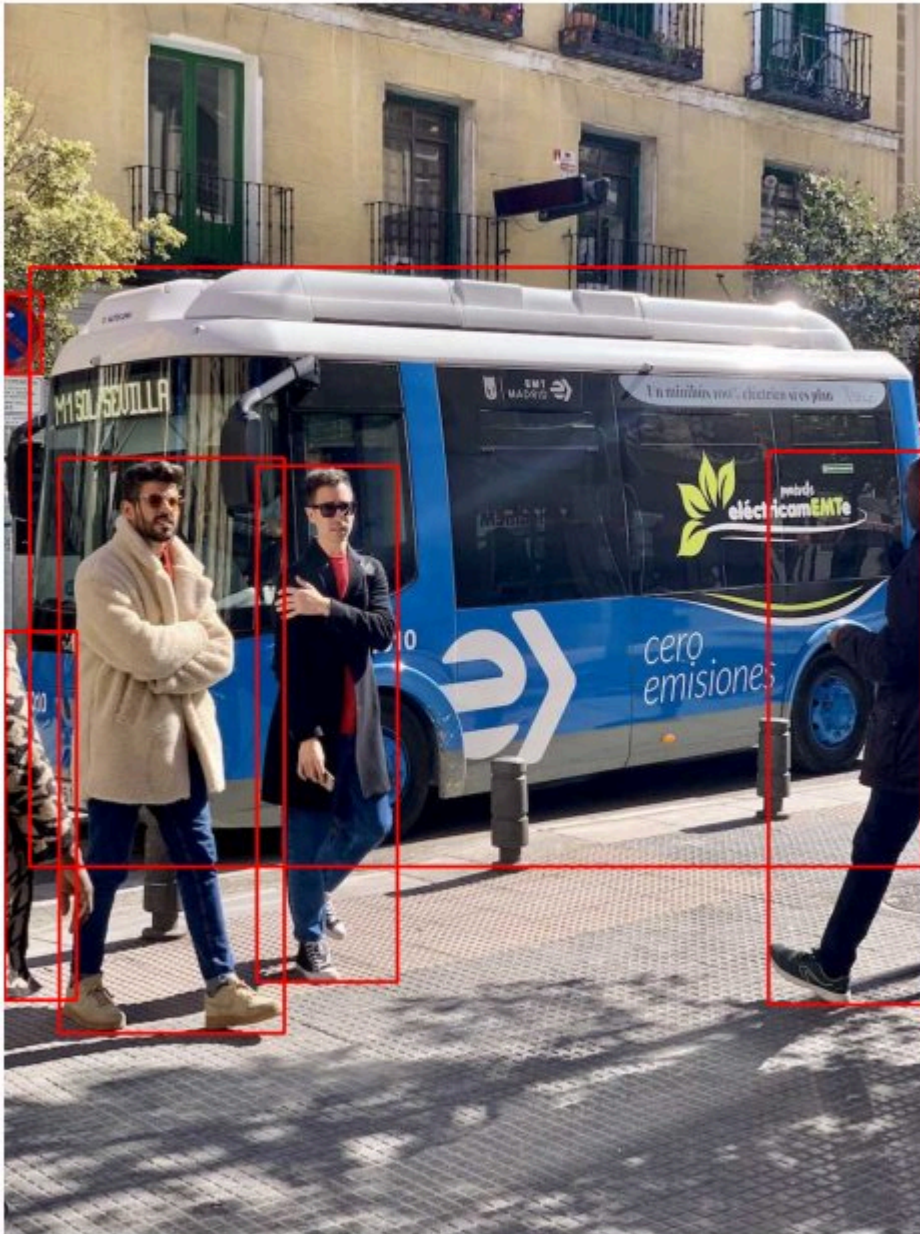
 **설명** — GPU에 있는 텐서를 `.cpu().numpy()` 로 NumPy 배열로 옮긴다. OpenCV/matplotlib은 GPU 텐서를 직접 다루지 못한다.

```
In [32]: import matplotlib.pyplot as plt

         for box in xyxy_coordinate:
             x1, y1, x2, y2 = map(int, box)
             cv2.rectangle(img_copy, (x1, y1), (x2, y2), color=(255, 0, 0), thickness=2)

         # 시각화
         plt.figure(figsize=(10, 8))
         plt.imshow(img_copy)
         plt.title("YOLO xyxy Bounding Boxes")
         plt.axis("off")
         plt.show()
```

## YOLO xyxy Bounding Boxes



 **설명** — 모든 박스를 빨간 사각형으로 직접 그렸다. `cv2.rectangle` 은 정수 좌표가 필요해서 `int` 로 바꾼다. 이미지를 RGB로 읽었으므로 `(255, 0, 0)` 이 빨강이다. (OpenCV 기본인 BGR로 읽었다면 파랑이 된다.)

```
In [33]: results[0].boxes.data
```

```
Out[33]: tensor([[2.2853e+01, 2.3128e+02, 8.0501e+02, 7.5686e+02, 8.7323e-01, 5.0000e+00],
          [4.8546e+01, 3.9855e+02, 2.4534e+02, 9.0271e+02, 8.6572e-01, 0.0000e+00],
          [6.6947e+02, 3.9219e+02, 8.0972e+02, 8.7703e+02, 8.5275e-01, 0.0000e+00],
          [2.2151e+02, 4.0580e+02, 3.4497e+02, 8.5755e+02, 8.2528e-01, 0.0000e+00],
          [0.0000e+00, 5.5052e+02, 6.3039e+01, 8.7344e+02, 2.6076e-01, 0.0000e+00],
          [5.7797e-02, 2.5446e+02, 3.2557e+01, 3.2488e+02, 2.5521e-01, 1.1000e+01]], device='cuda:0')
```

```
In [34]: det = results[0].boxes.data.cpu().numpy()
```

```
In [35]: type(det)
```

```
Out[35]: numpy.ndarray
```

```
In [36]: type(det)
```

```
Out[36]: numpy.ndarray
```

```
In [37]: url = "https://raw.githubusercontent.com/ultralytics/ultralytics/main/ultralytics/cfg/datasets/coco.yaml"
```

```
# URL에서 YAML 텍스트 불러오기
response = requests.get(url)
yaml_content = response.text

# YAML 파싱
parsed = yaml.safe_load(yaml_content)

# 클래스 이름 추출
class_names = parsed['names']
```

```
In [38]: class_names
```

```
Out[38]: {0: 'person',
 1: 'bicycle',
 2: 'car',
 3: 'motorcycle',
 4: 'airplane',
 5: 'bus',
 6: 'train',
 7: 'truck',
 8: 'boat',
 9: 'traffic light',
10: 'fire hydrant',
11: 'stop sign',
12: 'parking meter',
13: 'bench',
14: 'bird',
... (50줄 생략) ...
65: 'remote',
66: 'keyboard',
67: 'cell phone',
68: 'microwave',
69: 'oven',
70: 'toaster',
71: 'sink',
72: 'refrigerator',
73: 'book',
74: 'clock',
75: 'vase',
76: 'scissors',
77: 'teddy bear',
78: 'hair drier',
79: 'toothbrush'}
```

데이터 클래스를 확인하였다 -> 시나리오 기반으로 생각할때는 필요없는 클래스들이 많아 보인다 => 액션

 **설명** — COCO 80개 클래스 중 제조 현장 시나리오에 필요한 것은 거의 없다. 필요한 클래스만 **걸러서** 쓰거나, 원하는 대상을 **새로 학습**해야 한다. 아래는 먼저 "걸러 쓰기"를 해본다.

```
In [39]: img_copy = img_np.copy()

for det in results[0].boxes.data:
    x1, y1, x2, y2, conf, cls = det.tolist()
    if int(cls) == 0:
        x1, y1, x2, y2 = map(int, [x1, y1, x2, y2])
        label = f"{class_names[int(cls)]} {conf:.2f}"
        cv2.rectangle(img_copy, (x1, y1), (x2, y2), (255, 0, 0), 2)
        cv2.putText(img_copy, label, (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX,
                    0.9, (255, 0, 0), 2)

plt.figure(figsize=(10, 8))
plt.imshow(img_copy)
plt.title("Only Class 0 (person) Bounding Boxes with Labels")
plt.axis("off")
plt.show()
```

## Only Class 0 (person) Bounding Boxes with Labels



반쯤 가려진 사람도 잘 찾았다. 버스는 찾지않아도 되니까 결과 시각화에서 제외한다. 목표(시나리오)에 따르면 표지판에 대해서 찾을 수 있어야한다. -> 어떤 작업이 필요한가?

 **설명** — `boxes.data` 의 각 행을 `x1, y1, x2, y2, conf, cls` 로 풀고, `cls == 0` (person)인 박스만 이름·신뢰도와 함께 그렸다. 왼쪽 끝의 반쯤 잘린 사람도 0.26으로 잡혔다. 반면 표지판(stop sign)은 신뢰도가 낮고, COCO에 없는 표지판은 아예 찾을 수 없다. → 찾고 싶은 대상을 라벨링한 데이터로 추가 학습(파인튜닝) 이 필요하다 (Part 2).

```
In [40]: import os
bus_class_id = [k for k, v in class_names.items() if v == 'bus'][0]
os.makedirs("bus_crops", exist_ok=True)

count = 0
for det in results[0].boxes.data:
    x1, y1, x2, y2, conf, cls = det.tolist()
    if int(cls) == bus_class_id:
        x1, y1, x2, y2 = map(int, [x1, y1, x2, y2])
        cropped = img_np[y1:y2, x1:x2]
        save_path = f"bus_crops/bus_crop_{count}.jpg"
        cv2.imwrite(save_path, cv2.cvtColor(cropped, cv2.COLOR_RGB2BGR))
        print(f"버스 {count}번 잘라서 저장: {save_path}")
        count += 1
```

버스 0번 잘라서 저장: bus\_crops/bus\_crop\_0.jpg

 **설명** — 클래스 이름에서 bus의 번호(5)를 찾아, 버스 박스 영역만 배열 슬라이싱( `img_np[y1:y2, x1:x2]` )으로 잘라 저장한다. `cv2.imwrite`는 BGR 순서로 저장하므로 RGB → BGR 변환이 필요하다. 제조 라인에서는 이렇게 탐지 → 잘라내기 → 정밀 검사의 2단계로 많이 쓴다.

```
In [41]: import os
import cv2
import matplotlib.pyplot as plt

crop_folder = "bus_crops"

image_files = sorted([f for f in os.listdir(crop_folder) if f.endswith(".jpg")])

num_images = len(image_files)
plt.figure(figsize=(5 * num_images, 5))

for i, filename in enumerate(image_files):
    img_path = os.path.join(crop_folder, filename)
    img = cv2.imread(img_path)
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # OpenCV는 BGR로 불러오기 때문에 RGB로 변환

    plt.subplot(1, num_images, i + 1)
    plt.imshow(img)
    plt.title(filename)
    plt.axis("off")

plt.tight_layout()
plt.show()
```

bus\_crop\_0.jpg



 **설명** — 저장한 crop 이미지를 다시 읽어 확인한다. 버스 한 대가 잘려 나왔다.

## 개인 실험 1 — 사전학습 모델을 더 알아보기

기본 실습에서 생긴 질문을 직접 확인한다.

1. **신뢰도 임계값(conf)** 을 바꾸면 탐지 결과가 어떻게 달라지나?
2. **추가 샘플 이미지**에서도 잘 되나? **모델 크기·버전**(YOLOv8n / YOLOv8s / YOLO11n)에 따라 무엇이 달라지나?
3. **NMS의 IoU 임계값**은 겹친 박스에 어떤 영향을 주나?

```
In [42]: import pandas as pd
import torch
from pathlib import Path

DEVICE = 0 if torch.cuda.is_available() else "cpu"
bus_bgr = cv2.cvtColor(img_np, cv2.COLOR_RGB2BGR)

rows = []
for c in [0.05, 0.1, 0.25, 0.4, 0.55, 0.7, 0.85]:
    r = model.predict(bus_bgr, conf=c, device=DEVICE, verbose=False)[0]
    names = [class_names[int(k)] for k in r.boxes.cls]
    rows.append({"conf": c, "boxes": len(names), "person": names.count("person"),
                "bus": names.count("bus"), "others": ", ".join(n for n in names if n not in ("person", "bus")) or "-"},
               )
pd.DataFrame(rows)
```

Out[42]:

|   | conf | boxes | person | bus | others                                           |
|---|------|-------|--------|-----|--------------------------------------------------|
| 0 | 0.05 | 13    | 5      | 3   | stop sign, skateboard, bicycle, handbag, handbag |
| 1 | 0.10 | 6     | 4      | 1   | stop sign                                        |
| 2 | 0.25 | 6     | 4      | 1   | stop sign                                        |
| 3 | 0.40 | 4     | 3      | 1   | -                                                |
| 4 | 0.55 | 4     | 3      | 1   | -                                                |
| 5 | 0.70 | 4     | 3      | 1   | -                                                |
| 6 | 0.85 | 3     | 2      | 1   | -                                                |

### 실험 1-1 결과 해석

- **0.05:** 박스가 13개로 늘었다. 사람 5명, 버스 3개(같은 버스에 박스가 겹쳐 나옴)에 skateboard-bicycle-handbag 같은 **없는 물체(오탐)** 가 섞였다.
- **0.10~0.25:** 6개(버스 1, 사람 4, 정지 표지판 1)로 안정적이다. 과제 기본값 0.25가 무난한 이유다.
- **0.40~0.70:** 4개. 신뢰도 0.26짜리였던 **왼쪽 끝의 잘린 사람**과 **정지 표지판**이 빠진다.
- **0.85:** 3개. 확실한 사람 한 명(0.83)까지 빠진다.

→ 임계값은 **농침(미탐)**과 **헛것(오탐)** 사이의 조절 손잡이다. 제조 검사처럼 놓치면 안 되는 상황에서는 낮게, 오탐 비용이 크면 높게 잡아야 한다.

In [43]:

```

SAMPLES = {"bus": "https://ultralytics.com/images/bus.jpg",
            "zidane": "https://ultralytics.com/images/zidane.jpg"}
MODELS = ["yolov8n.pt", "yolov8s.pt", "yolo11n.pt"]

samples = {k: cv2.imdecode(np.frombuffer(requests.get(u).content, np.uint8), cv
2.IMREAD_COLOR) for k, u in SAMPLES.items()}
rows, vis = [], {}
for mname in MODELS:
    m = YOLO(mname)
    for sname, im in samples.items():
        m.predict(im, device=DEVICE, verbose=False)           # 첫 호출(워밍업)
        # 시간 측정에서 제외
        r = m.predict(im, conf=0.25, device=DEVICE, verbose=False)[0]
        names = [r.names[int(k)] for k in r.boxes.cls]
        rows.append({"model": mname, "image": sname, "boxes": len(names),
                     "detected": ", ".join(f"{n}×{names.count(n)}" for n in sorted(
set(names))),
                     "mean_conf": round(float(r.boxes.conf.mean()), 3) if len(na
mes) else 0,
                     "infer_ms": round(r.speed["inference"], 1),
                     "params(M)": round(sum(p.numel() for p in m.model.parameter
s()) / 1e6, 1)})
        vis[(mname, sname)] = r.plot()[ :, :, :-1]
model_cmp = pd.DataFrame(rows)
model_cmp

```

Out[43]:

|   | model      | image  | boxes | detected                     | mean_conf | infer_ms | params(M) |
|---|------------|--------|-------|------------------------------|-----------|----------|-----------|
| 0 | yolov8n.pt | bus    | 6     | bus×1, person×4, stop sign×1 | 0.655     | 10.3     | 3.2       |
| 1 | yolov8n.pt | zidane | 3     | person×2, tie×1              | 0.649     | 13.4     | 3.2       |
| 2 | yolov8s.pt | bus    | 5     | bus×1, person×4              | 0.838     | 14.6     | 11.2      |
| 3 | yolov8s.pt | zidane | 3     | person×2, tie×1              | 0.830     | 11.6     | 11.2      |
| 4 | yolo11n.pt | bus    | 5     | bus×1, person×4              | 0.837     | 14.6     | 2.6       |
| 5 | yolo11n.pt | zidane | 3     | person×2, tie×1              | 0.691     | 11.2     | 2.6       |

```
In [44]: fig, axes = plt.subplots(len(SAMPLES), len(MODELS), figsize=(5 * len(MODELS),
4.5 * len(SAMPLES)))
for i, sname in enumerate(SAMPLES):
    for j, mname in enumerate(MODELS):
        axes[i, j].imshow(vis[(mname, sname)]); axes[i, j].set_title(f"{mname}
| {sname}"); axes[i, j].axis("off")
plt.tight_layout(); plt.show()
```



## 실험 1-2 결과 해석

- 추가 샘플(zidane) 에서는 세 모델 모두 사람 2명 + 넥타이 1개로 같은 결과를 냈다.
- 버스 사진에서는 YOLOv8n만 정지 표지판(0.26)을 잡았고, YOLOv8s-YOLO11n은 버스 1 + 사람 4만 찾았다. 대신 평균 신뢰도가 v8n 0.66 → v8s-11n 0.84로 더 확신 있게 예측했다.
- 파라미터 수는 v8s(11.2M)가 v8n(3.2M)의 약 3.5배이고, YOLO11n(2.6M)은 v8n보다 오히려 작다.
- 한 장씩 추론한 시간(10~18ms)은 GPU 준비 시간의 영향이 커서, 이 실험만으로 속도 차이를 말하기는 어렵다. 속도 비교는 test 193장을 평균 낸 개인 실험 2-1에서 다시 본다.

→ 모델이 커지거나 최신 버전이라고 찾는 물체 수가 늘지는 않았고, 같은 물체를 더 확신 있게 맞히는 쪽으로 차이가 났다.

```
In [45]: rows = []
for iou in [0.3, 0.5, 0.7, 0.9]:
    for sname, im in samples.items():
        r = model.predict(im, conf=0.1, iou=iou, device=DEVICE, verbose=False)
        rows.append({"iou(NMS)": iou, "image": sname, "boxes": len(r.boxes)})
pd.DataFrame(rows).pivot(index="iou(NMS)", columns="image", values="boxes")
```

Out[45]:

|          | image | bus | zidane |
|----------|-------|-----|--------|
| iou(NMS) |       |     |        |
| 0.3      | 6     | 6   |        |
| 0.5      | 6     | 8   |        |
| 0.7      | 6     | 12  |        |
| 0.9      | 9     | 13  |        |

 **설명** — NMS(Non-Maximum Suppression)는 같은 물체에 겹쳐 나온 박스 중 신뢰도가 가장 높은 것만 남긴다. `iou` 값보다 많이 겹치는 박스를 지우기 때문에, **값이 클수록 겹친 박스를 더 많이 살려 둔다.**

### 실험 1-3 결과 해석

- **버스 사진**은 IoU 0.3~0.7에서 6개로 그대로였고, 0.9에서야 9개로 늘었다. 사람들이 서로 많이 겹치지 않아서다.
- **zidane 사진**은 0.3 → 6개, 0.5 → 8개, 0.7 → 12개, 0.9 → 13개로 크게 변했다. 같은 사람에게 여러 박스가 겹쳐 나오는데, IoU 기준이 느슨할수록 그 중복 박스가 지워지지 않고 남는다.

→ 제품이 **쭈뼌히 붙어 있는 라인**에서는 IoU를 너무 낮추면 붙은 제품 두 개가 하나로 합쳐지고, 너무 높이면 한 제품에 박스가 여러 개 생긴다. 기본값 0.7 근처에서 현장 배치에 맞게 조정해야 한다.

---

## Part 2. YOLOv8 학습 실습

### 데이터셋 다운로드

<https://universe.roboflow.com/warisara-kaewsuwan-cf2hs/stamp-bcrhe>

- 사용 버전: **v10** (2022-04-02) · YOLOv8 형식 · 클래스 `shoes`, `stamp`
- 분할: train 1,100 / valid 273 / test 193 · 416×416으로 리사이즈됨
- 시나리오: 작업대 위에서 신발에 **도장(stamp)** 이 찍혔는지 위에서 내려다보며 확인하는 제조 공정 영상

In [46]:

```
from ultralytics import YOLO
import os

model = YOLO("yolov8n.pt")
```

 **설명** — Part 1에서 쓰던 모델과 별개로, 학습을 시작할 **새 사전학습 모델**을 불러온다. COCO로 이미 학습된 가중치에서 출발하면(전이학습) 적은 데이터로도 빨리 수렴한다.

- datasets
  - stamp
    - test
    - train
    - valid
    - data.yaml

 **설명** — 과제와 같은 구조로 CV-YOLO/datasets/stamp/ 에 압축을 풀었다. 노트북이 CV-YOLO/notebooks/ 에 있으므로 과제 코드의 ../datasets/stamp/... 경로가 그대로 맞는다. 각 split 폴더에는 images/ (사진)와 labels/ (사진마다 클래스 x y w h 정규화 좌표가 적힌 txt)가 있다.

```
In [47]: model.train(
          data="../datasets/stamp/data.yaml",
          epochs=20,
          imgsz=640,
          batch=16,
          name="stamp_detect",
          device=0
        )
```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

... (106줄 생략) ...

| Epoch | GPU_mem | box_loss | cls_loss | dfl_loss | Instances | Size  |
|-------|---------|----------|----------|----------|-----------|-------|
| all   |         | 273      | 952      | 0.955    | 0.986     | 0.983 |

0.617

20 epochs completed in 0.064 hours.

Optimizer stripped from C:\Users\LOQ5060\Desktop\CV-YOLO\notebooks\runs\detect\stamp\_detect\weights\last.pt, 6.2MB

Optimizer stripped from C:\Users\LOQ5060\Desktop\CV-YOLO\notebooks\runs\detect\stamp\_detect\weights\best.pt, 6.2MB

Validating C:\Users\LOQ5060\Desktop\CV-YOLO\notebooks\runs\detect\stamp\_detect\weights\best.pt...

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs

|       | all | box_loss | cls_loss | dfl_loss | Instances | Size  |
|-------|-----|----------|----------|----------|-----------|-------|
| 0.638 |     | 273      | 952      | 0.952    | 0.987     | 0.985 |

|       |       |     |     |       |       |       |
|-------|-------|-----|-----|-------|-------|-------|
| 0.717 | shoes | 204 | 408 | 0.928 | 0.974 | 0.975 |
|-------|-------|-----|-----|-------|-------|-------|

|      |       |     |     |       |   |       |
|------|-------|-----|-----|-------|---|-------|
| 0.56 | stamp | 273 | 544 | 0.977 | 1 | 0.995 |
|------|-------|-----|-----|-------|---|-------|

0.56

Speed: 0.4ms preprocess, 1.5ms inference, 0.0ms loss, 1.5ms postprocess per image

Results saved to C:\Users\LOQ5060\Desktop\CV-YOLO\notebooks\runs\detect\stamp\_detect

Out[47]: ultralytics.utils.metrics.DetMetrics object with attributes:

```
ap_class_index: array([0, 1])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x000002B2A4FDAB50>
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']
curves_results: [[array([
0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.008008, 0.009009,
0.01001, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.021021, 0.022022, 0.023023,
0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032, 0.033033, 0.034034, 0.035035, 0.036036, 0.037037, 0.038038, 0.039039, 0.04004, 0.041041, 0.042042, 0.043043, 0.044044, 0.045045, 0.046046, 0.047047,
... (166줄 생략) ...
[1, 1, 1, ..., 0, 0]], shape=(2, 1000)), 'Confidence', 'Recall']]
fitness: 0.638485398537328
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.71738, 0.55959])
names: {0: 'shoes', 1: 'stamp'}
nt_per_class: array([408, 544])
nt_per_image: array([204, 273])
results_dict: {'metrics/precision(B)': 0.9521953850427981, 'metrics/recall(B)': 0.987171399571973, 'metrics/mAP50(B)': 0.9849713795038824, 'metrics/mAP50-95(B)': 0.638485398537328, 'fitness': 0.638485398537328}
save_dir: WindowsPath('C:/Users/LOQ5060/Desktop/CV-YOLO/notebooks/runs/detect/stamp_detect')
speed: {'preprocess': 0.42180915771652655, 'inference': 1.4938516482615318, 'loss': 0.00033956046656265367, 'postprocess': 1.5260494505387865}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}]
```

 **설명** — 사전학습 가중치에서 시작해 stamp 데이터로 파인튜닝한다.

| 인자                     | 의미                                                                                                            |
|------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>data</code>      | 데이터 설정 파일. 이미지 경로( <code>train / val / test</code> )와 클래스 이름( <code>names: ['shoes', 'stamp']</code> )이 들어 있다 |
| <code>epochs=20</code> | 학습 데이터 전체를 20번 반복                                                                                             |
| <code>imgsz=640</code> | 입력 크기. 원본(416×416)을 640으로 키워 넣는다                                                                              |
| <code>batch=16</code>  | 한 번에 16장씩 묶어서 학습                                                                                              |
| <code>name</code>      | 결과 폴더 이름 → <code>runs/detect/stamp_detect</code>                                                              |
| <code>device=0</code>  | 첫 번째 GPU 사용                                                                                                   |

- 로그 첫 부분의 `Overriding model.yaml nc=80 with nc=2` : COCO 80개 클래스 용 출력층을 **2개 클래스용으로 바꿔** 새로 학습한다는 뜻이다.
- 에폭마다 `box_loss` (위치 오차), `cls_loss` (분류 오차), `dfl_loss` (박스 경계 분포 오차)가 줄어드는지, 검증 mAP가 오르는지 본다.
- 마지막에 가장 좋았던 에폭의 가중치를 `weights/best.pt` 로 저장하고, 그 가중치로 한 번 더 검증한다.
- 셀의 `Out` 은 `model.train()` 이 돌려준 평가 객체다. PR 곡선용 숫자까지 모두 출력돼 매우 길다 (PDF에서는 앞뒤만 남겼다).

```
In [48]: metrics = model.val()
print("🔍 평가 결과:", metrics)
```

```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)
Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access (ping: 0.10.0 ms, read: 154.028.4 MB/s, size: 26.6 KB)
val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\datasets\stamp\valid\labels.cache... 273 images, 0 backgrounds, 0 corrupt: 100% 273/273 0.0s
      all      273      952      0.953      0.987      0.985
0.64
      shoes      204      408      0.928      0.974      0.975
0.717
      stamp      273      544      0.978      1      0.995
0.562
Speed: 2.0ms preprocess, 5.2ms inference, 0.0ms loss, 1.4ms postprocess per image
Results saved to C:\Users\LOQ5060\Desktop\CV-YOLO\notebooks\runs\detect\val
🔍 평가 결과: ultralytics.utils.metrics.DetMetrics object with attributes:

```

```

ap_class_index: array([0, 1])
box: ultralytics.utils.metrics.Metric object
confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x000002B2A826AC50>
curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']
... (168줄 생략) ...
      [      1,      1,      1, ...,      0,      0,
0]], shape=(2, 1000)), 'Confidence', 'Recall'])
fitness: 0.6395495664152615
keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.71673, 0.56237])
names: {0: 'shoes', 1: 'stamp'}
nt_per_class: array([408, 544])
nt_per_image: array([204, 273])
results_dict: {'metrics/precision(B)': 0.9530530332488807, 'metrics/recall(B)': 0.9871390667515323, 'metrics/mAP50(B)': 0.9850004041920235, 'metrics/mAP50-95(B)': 0.6395495664152615, 'fitness': 0.6395495664152615}
save_dir: WindowsPath('C:/Users/LOQ5060/Desktop/CV-YOLO/notebooks/runs/detect/val')
speed: {'preprocess': 1.966254944715226, 'inference': 5.202001465387275, 'loss': 0.0033406595121250374, 'postprocess': 1.4232915747039117}
stats: {'tp': [], 'conf': [], 'pred_cls': [], 'target_cls': [], 'target_img': []}

```

📖 **설명** — 학습이 끝난 모델( `best.pt` )로 **검증 데이터(valid 273장)** 를 다시 평가한다. 학습 때 쓴 `data` 설정을 기억하고 있어서 인자 없이 호출할 수 있다. 로그 표의 `Images` 는 해당 클래스가 나온 사진 수, `Instances` 는 정답 박스 수다. `print(metrics)` 는 객체 전체를 출력하므로 길다 (PDF에서는 앞뒤만 남겼다).

```
In [49]: import pandas as pd
```

```
data = {
    "Class": ["all", "shoes", "stamp"],
    "Images": [273, 204, 273],
    "Instances": [952, 408, 544],
    "Precision": [0.953, 0.928, 0.978],
    "Recall": [0.987, 0.974, 1.000],
    "mAP@.5": [0.985, 0.975, 0.995],
    "mAP@.5:.95": [0.640, 0.717, 0.562],
}

df = pd.DataFrame(data)

print("### 📊 Evaluation Results Table (Markdown Format)\n")
print(df.to_markdown(index=False))

print("\n### 📖 Explanation of Each Column")
print("""
- **Class**: Object class name. "all" means aggregated results over all classes.
- **Images**: Number of images in which this class appears (or total evaluation
images for 'all').
- **Instances**: Total number of ground truth objects for the class.
- **Precision**: Proportion of correct positive predictions (TP / (TP + FP)).
- **Recall**: Proportion of ground truths that were correctly detected (TP / (TP
+ FN)).
- **mAP@.5**: Mean Average Precision at IoU threshold 0.5.
- **mAP@.5:.95**: mAP averaged over IoU thresholds from 0.5 to 0.95 in steps of
0.05 (more stringent metric).
""")
```

```
### 📊 Evaluation Results Table (Markdown Format)
```

| Class | Images | Instances | Precision | Recall | mAP@.5 | mAP@.5:.95 |
|-------|--------|-----------|-----------|--------|--------|------------|
| all   | 273    | 952       | 0.953     | 0.987  | 0.985  | 0.640      |
| shoes | 204    | 408       | 0.928     | 0.974  | 0.975  | 0.717      |
| stamp | 273    | 544       | 0.978     | 1      | 0.995  | 0.562      |

```
### 📖 Explanation of Each Column
```

```
- **Class**: Object class name. "all" means aggregated results over all classes.
- **Images**: Number of images in which this class appears (or total evaluation
images for 'all').
- **Instances**: Total number of ground truth objects for the class.
- **Precision**: Proportion of correct positive predictions (TP / (TP + FP)).
- **Recall**: Proportion of ground truths that were correctly detected (TP / (TP
+ FN)).
- **mAP@.5**: Mean Average Precision at IoU threshold 0.5.
- **mAP@.5:.95**: mAP averaged over IoU thresholds from 0.5 to 0.95 in steps of
0.05 (more stringent metric).
```

 **설명** — 바로 위 검증 로그의 숫자를 표로 옮겨 정리했다. (과제 코드와 같은 방식이고, 값은 이번 실행 결과로 바꿨다. 같은 표를 `metrics` 에서 자동으로 만드는 코드는 개인 실험 2-0에 있다.)

| 지표         | 쉽게 말하면                                       |
|------------|----------------------------------------------|
| Precision  | 모델이 "있다"고 한 것 중 진짜 비율 → 낮으면 <b>헛것을 봄(오탐)</b> |
| Recall     | 실제로 있는 것 중 찾아낸 비율 → 낮으면 <b>놓침(미탐)</b>        |
| mAP@.5     | 박스가 절반 이상만 겹쳐도 정답 → "위치를 대충 맞히나"             |
| mAP@.5:.95 | 0.5~0.95로 점점 엄격하게 → "박스를 딱 맞게 치나"            |

```
In [50]: metrics.box.map, metrics.box.map50, metrics.box.map75, metrics.box.maps
```

```
Out[50]: (np.float64(0.6395495664152615),
np.float64(0.9850004041920235),
np.float64(0.7395406571714536),
array([ 0.71673, 0.56237]))
```

 **설명** — 표의 값을 코드로 꺼내는 방법. `map` (mAP@.5:.95), `map50`, `map75` (IoU 0.75 기준), `maps` (클래스별 mAP@.5:.95 — shoes, stamp 순).

## 평가 결과 해석

|                                                | Precision | Recall       | mAP@.5       | mAP@.5:.95   |
|------------------------------------------------|-----------|--------------|--------------|--------------|
| <b>이번 실행</b> (RTX 5060, ultralytics 8.4) — all | 0.953     | 0.987        | 0.985        | 0.640        |
| shoes                                          | 0.928     | 0.974        | 0.975        | <b>0.717</b> |
| stamp                                          | 0.978     | <b>1.000</b> | <b>0.995</b> | <b>0.562</b> |
| 과제 예시 (A100, ultralytics 8.3) — all            | 0.979     | 0.903        | 0.982        | 0.656        |

- 환경과 라이브러리 버전이 달라도 **mAP@.5 0.98 / mAP@.5:.95 0.64~0.66** 으로 과제 예시와 같은 수준이 재현됐다.
- **stamp**는 mAP@.5가 0.995이고 Recall이 1.000으로 하나도 놓치지 않았다. 그런데 mAP@.5:.95는 0.562로 shoes(0.717)보다 낮다. 데이터 확인에서 예상한 대로 **작은 도장은 찾기는 잘 찾지만 박스 경계를 정확히 맞추기 어렵다**. mAP75(0.740)가 mAP50(0.985)보다 크게 낮은 것도 같은 이유다.
- **shoes**는 Precision 0.928로, 없는 곳을 신발로 잡는 오탐이 도장보다 많다.

```
In [51]: from PIL import Image
import matplotlib.pyplot as plt

image = Image.open('../datasets/stamp/test/images/clip1_mp4-33_jpg.rf.95be2fb4fb
ae44fb12fd600d1d966f76.jpg')

image
```

Out[51]: 2021-10-06 15:17:47  
PG03-LASTING-QC



```
In [52]: results = model.predict('../datasets/stamp/test/images/clip1_mp4-33_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg')
results[0].save(filename='predicted1.jpg')
```

image 1/1 C:\Users\LQ5060\Desktop\CV-YOLO\notebooks\..\datasets\stamp\test\images\clip1\_mp4-33\_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg: 640x640 2 shoess, 2 stamps, 14.5ms

Speed: 2.4ms preprocess, 14.5ms inference, 2.8ms postprocess per image at shape (1, 3, 640, 640)

Out[52]: 'predicted1.jpg'

```
In [53]: from PIL import Image
import matplotlib.pyplot as plt

image = Image.open("predicted1.jpg")

image
```

Out[53]:



In [54]: results[0].boxes

Out[54]: ultralytics.engine.results.Boxes object with attributes:

```
cls: tensor([0., 1., 1., 0.], device='cuda:0')
conf: tensor([0.9577, 0.9223, 0.8993, 0.4741], device='cuda:0')
data: tensor([[232.5940, 260.6025, 298.7854, 338.3451, 0.9577, 0.0000],
              [168.5933, 275.5515, 189.8084, 316.0340, 0.9223, 1.0000],
              [251.9562, 95.4230, 274.1241, 134.1069, 0.8993, 1.0000],
              [243.2766, 362.1186, 309.7711, 415.9079, 0.4741, 0.0000]], device='c
uda:0')
id: None
is_track: False
orig_shape: (416, 416)
shape: torch.Size([4, 6])
xywh: tensor([[265.6897, 299.4738, 66.1914, 77.7425],
              [179.2008, 295.7927, 21.2151, 40.4825],
              [263.0402, 114.7650, 22.1678, 38.6839],
              [276.5239, 389.0132, 66.4945, 53.7892]], device='cuda:0')
xywhn: tensor([[0.6387, 0.7199, 0.1591, 0.1869],
               [0.4308, 0.7110, 0.0510, 0.0973],
               [0.6323, 0.2759, 0.0533, 0.0930],
               [0.6647, 0.9351, 0.1598, 0.1293]], device='cuda:0')
xyxy: tensor([[232.5940, 260.6025, 298.7854, 338.3451],
              [168.5933, 275.5515, 189.8084, 316.0340],
              [251.9562, 95.4230, 274.1241, 134.1069],
              [243.2766, 362.1186, 309.7711, 415.9079]], device='cuda:0')
xyxyn: tensor([[0.5591, 0.6264, 0.7182, 0.8133],
               [0.4053, 0.6624, 0.4563, 0.7597],
               [0.6057, 0.2294, 0.6590, 0.3224],
               [0.5848, 0.8705, 0.7446, 0.9998]], device='cuda:0')
```

 **설명** — 학습에 쓰지 않은 **test 이미지**로 예측했다. `results[0].save()` 는 박스를 그린 이미지를 파일로 저장한다. `boxes` 의 `cls` 가 이제 0(shoes)과 1(stamp)뿐이다 → COCO 80개 클래스 모델이 우리 시나리오 전용 모델로 바뀌었다.

- 과제 예시와 같은 test 사진에서 **shoes 2개(0.96, 0.47) + stamp 2개(0.92, 0.90)** 를 찾았다. 과제 예시는 shoes 1 + stamp 2였다.
- 추가로 잡힌 신발(0.47)은 박스 좌표(y 362~416)를 보면 **이미지 아래 가장자리에 잘려 보이는 신발**이다. 신뢰도가 0.5 미만이라, 임계값을 0.5로 올리면 과제 예시와 같은 결과가 된다.

## 개인 실험 2 — 학습 결과를 자세히 보고, 성능 범위를 넓게 확인하기

| 실험  | 바꾼 것 / 본 것                                           | 확인하려는 것                     |
|-----|------------------------------------------------------|-----------------------------|
| 2-0 | 데이터 분포, 학습 곡선, 혼동행렬, 평가표 자동 생성                       | 학습이 잘 됐나? 어디서 틀리나?          |
| 2-1 | 모델·에폭 (YOLOv8n 20/50, <b>YOLO11n</b> , YOLOv8s)      | 오래 학습하면? 최신 모델이면? 큰 모델이면?   |
| 2-2 | <b>신뢰도 최적화</b> (valid에서 고르고 test로 확인)                | 기본값 0.25보다 나은 신뢰도가 있나?      |
| 2-3 | <b>추가 샘플 데이터</b> : test 193장 전체 + 조건 변화(어둡게·흐리게·노이즈) | 처음 보는 데이터와 나쁜 촬영 환경에서도 버티나? |
| 2-4 | 틀린 사례 모아보기                                           | 어떤 상황에서 틀리나?                |

2-1부터는 학습에 쓰지 않은 **test 셋** 기준으로 비교한다.

### 2-0. 학습 결과 자세히 보기

```
In [55]: import shutil

ROOT = Path.cwd().parent
STAMP = ROOT / "datasets" / "stamp"
RUNS = ROOT / "runs" / "stamp" # 비교 실험 결과 저장 위치
BASE_RUN = Path(model.trainer.save_dir) # 위에서 학습한 기본 모델(stamp_detect) 폴더
print("기본 학습 결과 폴더:", BASE_RUN)

# 실험용 데이터 설정: 절대경로로 고정해 어디서 실행해도 같은 데이터를 쓰게 한다
cfg = yaml.safe_load((STAMP / "data.yaml").read_text(encoding="utf-8"))
DATA = STAMP / "data_local.yaml"
DATA.write_text(yaml.safe_dump({"path": STAMP.as_posix(), "train": "train/images",
                                "val": "valid/images",
                                "test": "test/images", "names": cfg["names"]}, allow_unicode=True), encoding="utf-8")
```

기본 학습 결과 폴더: C:\Users\LOQ5060\Desktop\CV-YOLO\notebooks\runs\detect\stamp\_detect

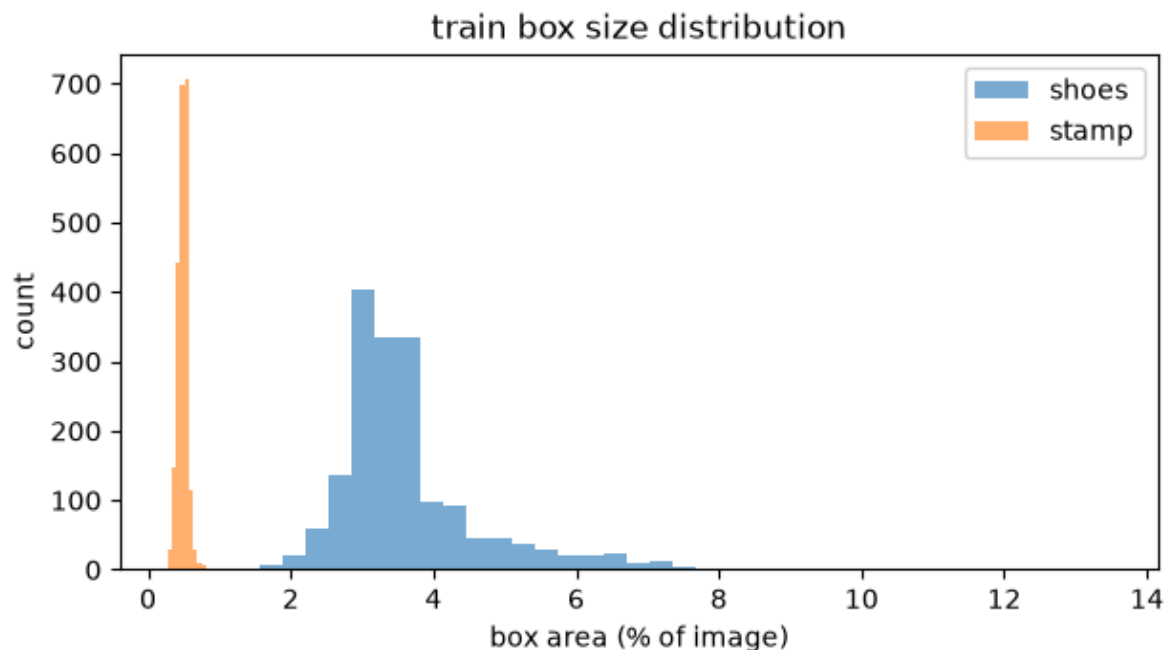
Out[55]: 133

```
In [56]: # 데이터 분포
def split_stats(split):
    labels = list((STAMP / split / "labels").glob("*.txt"))
    cls = [int(l.split()[0]) for f in labels for l in f.read_text().splitlines() if l.strip()]
    sizes = [float(l.split()[3]) * float(l.split()[4]) for f in labels for l in f.read_text().splitlines() if l.strip()]
    return {"split": split, "images": len(labels), "shoes": cls.count(0), "stamp": cls.count(1),
            "평균 박스 면적(%)": round(100 * np.mean(sizes), 2)}

def box_area(split, c):
    return [100 * float(l.split()[3]) * float(l.split()[4])
            for f in (STAMP / split / "labels").glob("*.txt") for l in f.read_text().splitlines() if l.strip() and int(l.split()[0]) == c]

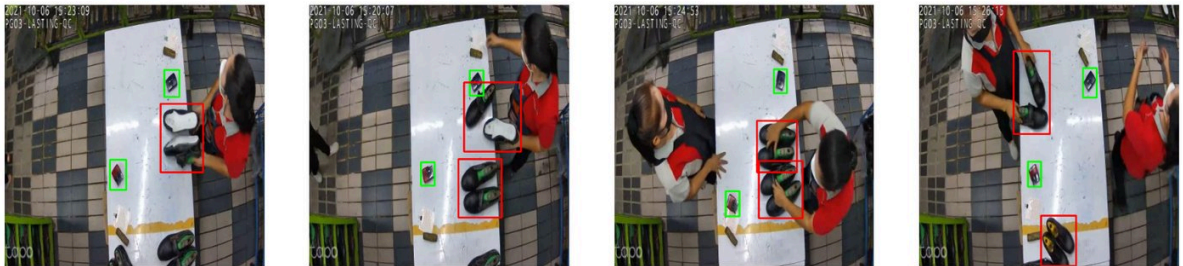
display(pd.DataFrame([split_stats(s) for s in ("train", "valid", "test")]))
plt.figure(figsize=(7, 3.5))
plt.hist(box_area("train", 0), bins=40, alpha=.6, label="shoes"); plt.hist(box_area("train", 1), bins=40, alpha=.6, label="stamp")
plt.xlabel("box area (% of image)"); plt.ylabel("count"); plt.legend(); plt.title("train box size distribution"); plt.show()
```

|   | split | images | shoes | stamp | 평균 박스 면적(%) |
|---|-------|--------|-------|-------|-------------|
| 0 | train | 1100   | 1750  | 2181  | 1.90        |
| 1 | valid | 273    | 408   | 544   | 1.59        |
| 2 | test  | 193    | 250   | 383   | 1.49        |



```
In [57]: samples_tr = sorted((STAMP / "train" / "images").glob("*"))[:275][:4]
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
for ax, p in zip(axes, samples_tr):
    im = cv2.cvtColor(cv2.imread(str(p)), cv2.COLOR_BGR2RGB); H, W = im.shape[:2]
    for line in (STAMP / "train" / "labels" / (p.stem + ".txt")).read_text().splitlines():
        c, cx, cy, bw, bh = map(float, line.split()[:5])
        cv2.rectangle(im, (int((cx-bw/2)*W), int((cy-bh/2)*H)), (int((cx+bw/2)*W), int((cy+bh/2)*H)),
            (255, 0, 0) if int(c) == 0 else (0, 255, 0), 2)
    ax.imshow(im); ax.axis("off")
plt.suptitle("train labels (red = shoes, green = stamp)"); plt.tight_layout();
plt.show()
```

train labels (red = shoes, green = stamp)

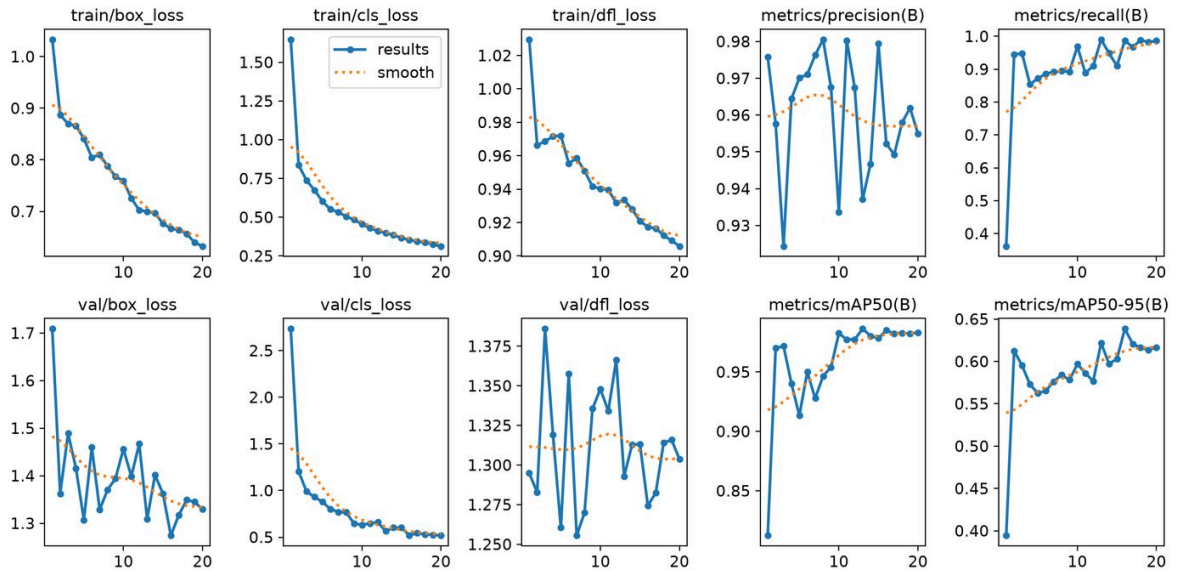


## 데이터 확인 결과

- train 1,100장에 **shoes 1,750개 / stamp 2,181개**로 두 클래스가 크게 치우치지 않았다. 사진 한 장에 도장이 평균 2개쯤 있다.
- 박스 하나가 이미지에서 차지하는 면적이 평균 **1.5~1.9%**로 매우 작다. 히스토그램을 보면 **stamp가 shoes보다 훨씬 작은 쪽에 몰려 있다**. 416×416 이미지에서 도장은 한 변이 20픽셀 안팎이다.
- 라벨 샘플을 보면 위에서 내려다본 작업대 영상이고, 작업자의 손·팔이 신발과 도장을 자주 가린다.

→ 예상: 작은 도장은 위치를 대충 맞추기는 쉽지만 박스를 딱 맞추기는 어렵고(mAP50-95 낮음), 가려진 신발은 놓치기 쉬울 것이다.

```
In [58]: display(Image.open(BASE_RUN / "results.png"))
```

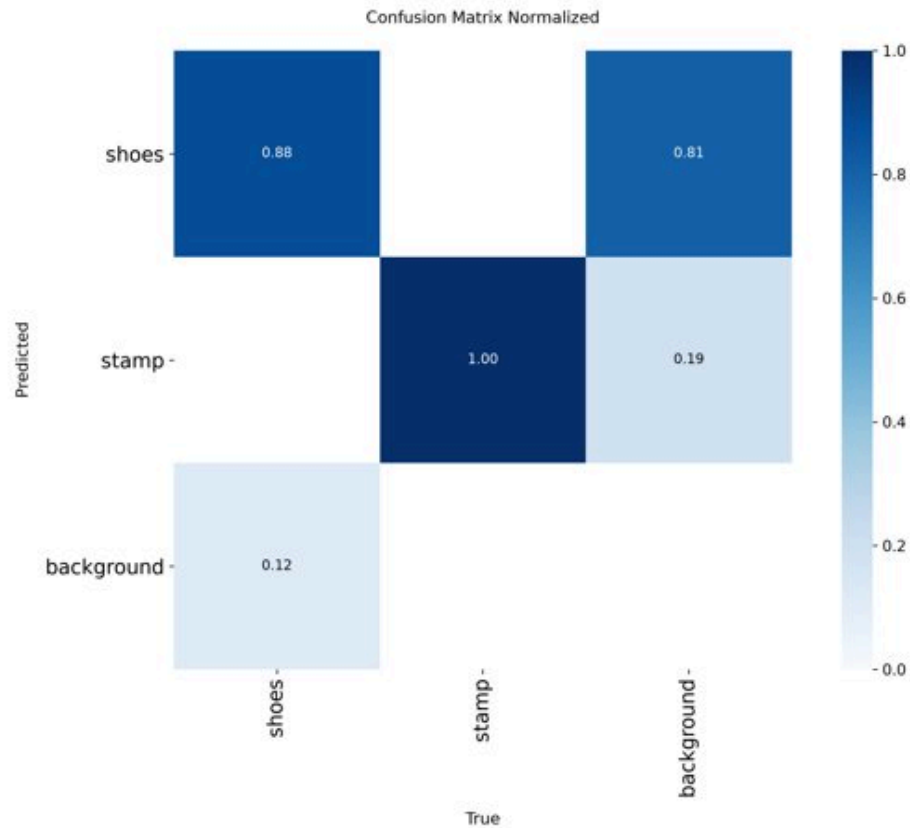


### 학습 곡선 해석

- 학습 손실( `train/box_loss` , `cls_loss` , `df_l_loss` )은 20 에폭 동안 **꾸준히 감소** 했다.
- 검증 mAP50은 10 에폭쯤 **0.98에서 포화**됐고, mAP50-95는 0.56~0.64 사이에서 오르내리다 **16 에폭에서 최고(0.638)** 였다. Ultralytics는 이 최고 시점을 `best.pt` 로 저장한다.
- 16 에폭 이후에는 0.61~0.62에 머문다.

→ 20 에폭이면 충분히 수렴했다. 더 학습해도 크게 좋아지지 않을 것으로 보이며, 이 예상은 실험 2-1(50 에폭)에서 확인한다.

```
In [59]: display(Image.open(BASE_RUN / "confusion_matrix_normalized.png").resize((640, 480)))
```



**설명** — 정규화된 혼동행렬. 세로가 정답, 가로가 예측이다. `background` 행·열은 "물체가 있는데 못 찾음", "없는데 찾음"을 뜻한다.

- **stamp**는 정답의 **100%** 를 stamp로 맞혔다.
- **shoes**는 **88%** 를 맞히고 **12%**는 **배경으로 놓쳤다**(background 행). 신발과 도장을 서로 헷갈린 경우는 없다.
- 오른쪽 background 열은 "실제로는 없는데 찾은 박스"가 어느 클래스로 나왔는지의 비율이다. **오탐의 81%가 shoes**였다. 위 평가표에서 shoes의 Precision이 더 낮았던 것과 같은 결과다.

→ 이 모델의 약점은 클래스 혼동이 아니라 **신발을 놓치거나 신발이 아닌 곳을 신발로 잡는 것**이다. 원인은 실험 2-4에서 틀린 사진을 직접 보며 확인한다.

```
In [60]: # 과제의 평가표를 metrics 객체에서 자동으로 만들기 (숫자를 손으로 옮기지 않아도 된다)
rows = [{"Class": "all", "Images": int(metrics.nt_per_image.max()), "Instances": int(metrics.nt_per_class.sum()),
        "Precision": metrics.box.mp, "Recall": metrics.box.mr, "mAP@.5": metrics.box.map50, "mAP@.5:.95": metrics.box.map}]
for i, c in enumerate(metrics.box.ap_class_index):
    p, r, ap50, ap = metrics.box.class_result(i)
    rows.append({"Class": model.names[int(c)], "Images": int(metrics.nt_per_image[c]), "Instances": int(metrics.nt_per_class[c]),
                "Precision": p, "Recall": r, "mAP@.5": ap50, "mAP@.5:.95": ap})
print(pd.DataFrame(rows).round(3).to_markdown(index=False))
```

```

| Class | Images | Instances | Precision | Recall | mAP@.5 | mAP@.
5:.95 |
|:-----|:-----:|:-----:|:-----:|:-----:|:-----:|:-----
---:|
| all | 273 | 952 | 0.953 | 0.987 | 0.985 | 0.
64 |
| shoes | 204 | 408 | 0.928 | 0.974 | 0.975 | 0.
717 |
| stamp | 273 | 544 | 0.978 | 1 | 0.995 | 0.
562 |

```

 **설명** — 과제 코드는 로그의 숫자를 손으로 옮겨 적었다. `metrics` 에서 직접 꺼내면 다시 학습해도 표가 자동으로 맞게 바뀌고, 옮겨 적다 틀릴 일도 없다. (`Images` 의 all 값은 전체 검증 사진 수 273장이다. 모든 사진에 stamp가 있어서 클래스별 최댓값과 같다.)

## 2-1. 모델·에폭 비교 — 모델을 바꾸면 성능이 오를까?

### 가설

|    | 바꾼 것                                | 예상      | 근거                                                        |
|----|-------------------------------------|---------|-----------------------------------------------------------|
| H1 | YOLOv8n → <b>YOLO11n</b><br>(다른 모델) | 성능<br>↑ | 더 최신 구조(C3k2 블록, C2PSA 어텐션)라 적은 파라미터로 특징을 더 잘 뽑는다고 알려져 있다 |
| H2 | YOLOv8n → <b>YOLOv8s</b><br>(큰 모델)  | 성능<br>↑ | 파라미터가 3.7배라 더 복잡한 패턴을 학습할 수 있다                            |
| H3 | 20 에폭 → <b>50 에폭</b>                | 성능<br>↑ | 더 오래 학습하면 더 수렴한다                                          |

학습 설정(데이터, 입력 크기 640, 배치 16)은 모두 같게 두고 **바꾼 것 하나씩만** 비교한다. 비교는 학습·선택에 쓰지 않은 **test** 셋으로 한다.

```

In [61]: EXPS = [("stamp_detect", BASE_RUN, "yolov8n.pt", 20),          # 위에서 학습한
기본 모델
                ("v8n_e50", RUNS / "v8n_e50", "yolov8n.pt", 50),
                ("11n_e20", RUNS / "11n_e20", "yolo11n.pt", 20),
                ("v8s_e20", RUNS / "v8s_e20", "yolov8s.pt", 20)]
RUN_DIR = {name: run for name, run, _, _ in EXPS}
summary = []
for name, run, weights, ep in EXPS:
    if not (run / "weights" / "best.pt").exists(): # 이미 학습한 실험은 결과
    재사용
        YOLO(weights).train(data=str(DATA), epochs=ep, imgsz=640, batch=16, dev
ice=DEVICE,
                                project=str(RUNS), name=name, exist_ok=True, workers
=2, seed=42, verbose=False)
        m = YOLO(str(run / "weights" / "best.pt"))
        v = m.val(data=str(DATA), split="test", device=DEVICE, verbose=False, plots
=False,
                    project=str(RUNS), name=f"{name}_test", exist_ok=True)
        res = pd.read_csv(run / "results.csv")
        summary.append({"exp": name, "model": weights, "epochs": ep, "best_epoch":
int(res.loc[res["metrics/mAP50-95(B)"].idxmax(), "epoch"]),
                        "params(M)": round(sum(p.numel() for p in m.model.parameters
()) / 1e6, 2),
                        "P": v.box.mp, "R": v.box.mr, "mAP50": v.box.map50, "mAP50-
95": v.box.map,
                        "shoes mAP50-95": v.box.maps[0], "stamp mAP50-95": v.box.map
s[1],
                        "infer_ms": v.speed["inference"]})
exp_df = pd.DataFrame(summary).round(3)
(ROOT / "results").mkdir(exist_ok=True)
exp_df.to_csv(ROOT / "results" / "stamp_experiments.csv", index=False)
exp_df

```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)  
 Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs  
 WARNING val: Slow image access detected (ping: 0.40.3 ms, read: 23.72.9 MB/s, size: 27.5 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>  
 val: New cache created: C:\Users\LOQ5060\Desktop\CV-YOLO\datasets\stamp\test\labels.cache

all 193 633 0.951 0.985 0.985

0.627

Speed: 1.5ms preprocess, 4.8ms inference, 0.0ms loss, 1.2ms postprocess per image  
 Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs  
 val: Fast image access (ping: 0.10.0 ms, read: 98.516.5 MB/s, size: 26.8 KB)  
 val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\datasets\stamp\test\labels.cache...  
 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.982 0.92 0.979

0.626

... (1줄 생략) ...

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

YOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.4 GFLOPs  
 val: Fast image access (ping: 0.10.0 ms, read: 106.820.7 MB/s, size: 27.0 KB)  
 val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\datasets\stamp\test\labels.cache...  
 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.975 0.931 0.982

0.634

Speed: 2.0ms preprocess, 7.1ms inference, 0.0ms loss, 1.6ms postprocess per image  
 Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

Model summary (fused): 72 layers, 11,126,358 parameters, 0 gradients, 28.4 GFLOPs  
 val: Fast image access (ping: 0.10.0 ms, read: 86.113.4 MB/s, size: 27.1 KB)  
 val: Scanning C:\Users\LOQ5060\Desktop\CV-YOLO\datasets\stamp\test\labels.cache...  
 193 images, 0 backgrounds, 0 corrupt: 100% 193/193 0.0s

all 193 633 0.962 0.939 0.975

0.628

Speed: 1.5ms preprocess, 6.6ms inference, 0.0ms loss, 1.7ms postprocess per image

Out[61]:

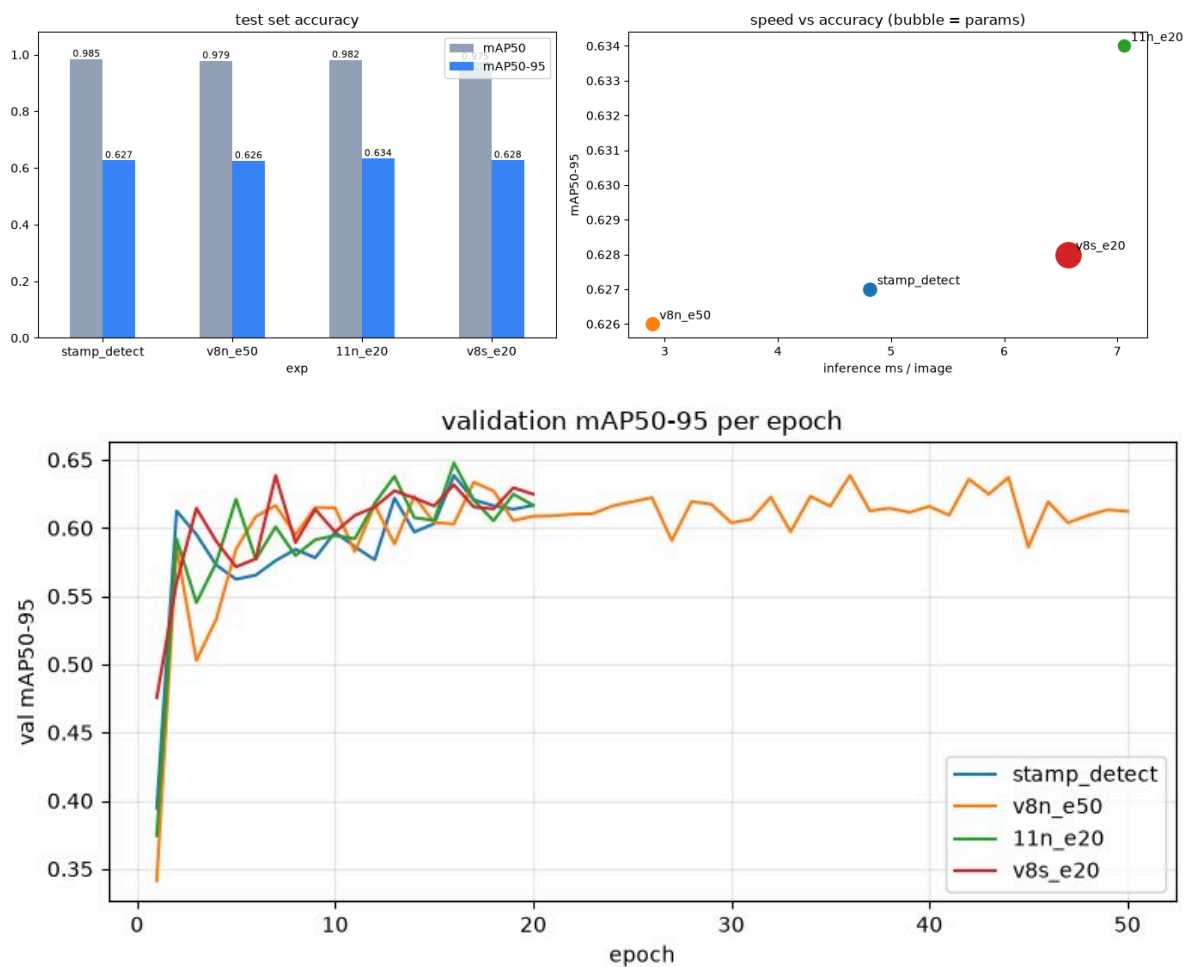
|   | exp          | model      | epochs | best_epoch | params(M) | P     | R     | mAP50 | mAP50-95 | shoes<br>mAP50-95 | stamp<br>mAP50-95 | infer_ms |
|---|--------------|------------|--------|------------|-----------|-------|-------|-------|----------|-------------------|-------------------|----------|
| 0 | stamp_detect | yolov8n.pt | 20     | 16         | 3.01      | 0.951 | 0.985 | 0.985 | 0.627    | 0.696             | 0.558             | 4.815    |
| 1 | v8n_e50      | yolov8n.pt | 50     | 36         | 3.01      | 0.982 | 0.920 | 0.979 | 0.626    | 0.700             | 0.553             | 2.890    |
| 2 | 11n_e20      | yolo11n.pt | 20     | 16         | 2.59      | 0.975 | 0.931 | 0.982 | 0.634    | 0.692             | 0.576             | 7.056    |
| 3 | v8s_e20      | yolov8s.pt | 20     | 7          | 11.14     | 0.962 | 0.939 | 0.975 | 0.628    | 0.693             | 0.564             | 6.568    |

```

In [62]: fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
exp_df.plot.bar(x="exp", y=["mAP50", "mAP50-95"], rot=0, ax=axes[0], color=["#9
4a3b8", "#3b82f6"])
axes[0].set_ylim(0, 1.08); axes[0].set_title("test set accuracy")
for c in axes[0].containers: axes[0].bar_label(c, fmt="%.3f", fontsize=8)
for _, r in exp_df.iterrows():
    axes[1].scatter(r["infer_ms"], r["mAP50-95"], s=r["params(M)"] * 40)
    axes[1].annotate(r["exp"], (r["infer_ms"], r["mAP50-95"]), textcoords="offse
t points", xytext=(6, 4))
axes[1].set_xlabel("inference ms / image"); axes[1].set_ylabel("mAP50-95"); axes
[1].set_title("speed vs accuracy (bubble = params)")
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_experiments.
png", dpi=150); plt.show()

# 에폭별 검증 mAP 곡선
plt.figure(figsize=(8, 4))
for name, run, _, _ in EXPS:
    res = pd.read_csv(run / "results.csv")
    plt.plot(res["epoch"], res["metrics/mAP50-95(B)"], label=name)
plt.xlabel("epoch"); plt.ylabel("val mAP50-95"); plt.legend(); plt.title("valida
tion mAP50-95 per epoch"); plt.grid(alpha=.3)
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_epoch_curve
s.png", dpi=150); plt.show()

```



## 실험 2-1 결과 해석

### 결과

| 실험                     | 파라미터         | test<br>mAP50 | test<br>mAP50-95 | shoes        | stamp        | 최고<br>에폭 | 가설            |
|------------------------|--------------|---------------|------------------|--------------|--------------|----------|---------------|
| YOLOv8n · 20ep<br>(기본) | 3.01M        | <b>0.985</b>  | 0.627            | 0.696        | 0.558        | 16       | -             |
| YOLO11n · 20ep         | <b>2.59M</b> | 0.982         | <b>0.634</b>     | 0.692        | <b>0.576</b> | 16       | H1:<br>+0.007 |
| YOLOv8s · 20ep         | 11.14M       | 0.975         | 0.628            | 0.693        | 0.564        | 7        | H2:<br>+0.001 |
| YOLOv8n · 50ep         | 3.01M        | 0.979         | 0.626            | <b>0.700</b> | 0.553        | 36       | H3:<br>-0.001 |

### 분석 — 세 가설 모두 뚜렷한 성능 향상은 없었다

- 네 조합의 test mAP50-95가 **0.626~0.634로 차이가 0.008**뿐이다. 같은 YOLOv8n 20 에폭 설정을 다시 학습했을 때도 0.637이 나왔다. 즉 **학습할 때마다  $\pm 0.01$  정도 흔들리는데, 모델 간 차이가 그보다 작다.**
- H1(YOLO11n): 가장 높았지만 차이가 흔들림 수준이다. 다만 **파라미터가 14% 적는데 같은 성능**을 냈고, 도장(0.558 → 0.576)은 가장 좋았다.
- H2(YOLOv8s): 파라미터가 3.7배인데 이득이 없다. 최고 성능이 **7 에폭**에 나오고 이후엔 오르지 않았다. 큰 모델이 작은 데이터에서 일찍 한계에 닿았다.
- H3(50 에폭): 에폭별 곡선에서 20 에폭 이후 0.60~0.64 사이를 오르내리기만 한다.

### 차이가 없는 이유

1. 이미 거의 다 맞히고 있다. mAP50이 네 모델 모두 0.975~0.985다. "찾기"는 끝난 상태라 모델을 바꿔도 올라갈 여지가 거의 없다. 남은 여지는 박스를 얼마나 딱 맞게 치느냐(mAP50-95)뿐이다.
2. 남은 오차는 모델이 아니라 **박스 크기·라벨의 한계**다. 도장은 416×416 사진에서 한 변이 20픽셀 안팎이다(2-0 박스 크기 분포). 박스가 몇 픽셀만 어긋나도 IoU가 크게 떨어진다. 게다가 정답 박스도 사람이 그은 것이라 몇 픽셀 오차가 있다. 모델이 좋아져도 **라벨보다 더 정확하게 맞출 수는 없다**. 모든 모델에서 stamp mAP50-95(0.55~0.58)가 shoes(0.69~0.70)보다 낮은 이유다.
3. **틀리는 사진은 데이터에 드문 상황**이다. 2-4에서 보듯 틀리는 사진은 가장자리에 잘린 신발, 손에 들린 신발이다. 이런 장면은 학습 데이터에 적어서 모델 구조를 바꿔도 배울 수 없다.
4. **데이터가 작고 서로 비슷하다**. train 1,100장은 거의 전부 같은 작업대를 위에서 찍은 영상 2개(595장 + 496장)에서 뽑은 프레임이라, 연속된 장면이 많다. 이 정도 데이터는 nano 모델로도 충분히 학습된다. 큰 모델의 추가 용량은 쓸 곳이 없다.

→ **결론**: 이 과제에서 성능을 올리려면 모델을 바꾸는 것보다 **데이터를 바꿔야 한다**(가장자리·손에 든 신발 사진 추가, 더 높은 해상도로 촬영, 라벨 경계 정밀화). 모델을 고른다면 성능은 같고 가장 가벼운 **YOLO11n**이 현장 적용에 유리하다. 이후 실험(2-2~2-4)은 YOLO11n으로 진행했다.

## 2-2. 신뢰도 최적화

기본 코드는 예측할 때 신뢰도 기본값(0.25)을 그대로 쓴다. 1번 실험처럼 신뢰도를 바꿔가며 **가장 균형 좋은 값**을 찾는다.

**가설 (H4):** 기본값 0.25는 COCO 같은 일반 데이터 기준이므로, 이 데이터에 맞게 신뢰도를 고르면 오탐이 줄어 **F1과 개수 정확도가 좋아질 것이다.**

- **valid** 셋에서 신뢰도별 F1(정밀도·재현율의 조화평균)을 계산해 최적값을 고르고,
- 학습·선택에 쓰지 않은 **test** 셋에서 기본값 0.25와 비교한다.

예측 박스를 정답 박스와 IoU 0.5 이상으로 짝지어 TP(맞힘)·FP(헛것)·FN(놓침)을 직접 센다.

```

In [63]: best_name = exp_df.sort_values("mAP50-95", ascending=False).iloc[0]["exp"]
best_model = YOLO(str(RUN_DIR[best_name] / "weights" / "best.pt")) # 2-1에서
가장 좋았던 모델로 진행
print("가장 좋은 모델:", best_name)

from ultralytics.utils.metrics import box_iou

def collect(model, split):
    """사진마다 정답 박스와 예측 박스(신뢰도 0.01 이상 전부)를 모아 둔다."""
    recs = []
    for r in model.predict(str(STAMP / split / "images"), conf=0.01, device=DEV
ICE, stream=True, verbose=False):
        h, w = r.orig_shape
        rows = [list(map(float, l.split())) for l in (STAMP / split / "labels"
/ (Path(r.path).stem + ".txt")).read_text().splitlines() if l.strip()]
        gt = torch.tensor([(cx - bw / 2) * w, (cy - bh / 2) * h, (cx + bw / 2)
* w, (cy + bh / 2) * h] for _, cx, cy, bw, bh in rows]).reshape(-1, 4)
        recs.append((torch.tensor([int(r[0]) for r in rows]), gt, r.bboxes.cls.c
pu(), r.bboxes.xyxy.cpu(), r.bboxes.conf.cpu()))
    return recs

def score(recs, conf):
    """신뢰도 conf 이상 박스만 남겼을 때의 정밀도·재현율·F1, 개수가 틀린 사진
수"""
    tp = fp = fn = wrong = 0
    for gc, gb, pc, pb, pf in recs:
        keep = pf >= conf
        pc, pb, pf = pc[keep], pb[keep], pf[keep]
        used = set()
        for i in pf.argsort(descending=True).tolist(): # 신뢰도 높은 박
스부터 정답과 짝짓기
            ious = box_iou(pb[i:i + 1], gb)[0] if len(gb) else torch.tensor([])
            cand = [(float(ious[j]), j) for j in range(len(gb)) if j not in use
d and gc[j] == pc[i] and ious[j] >= 0.5]
            if cand:
                used.add(max(cand)[1]); tp += 1
            else:
                fp += 1
        fn += len(gb) - len(used)
        wrong += any((pc == k).sum() != (gc == k).sum() for k in (0, 1))
    p, r = tp / max(tp + fp, 1), tp / max(tp + fn, 1)
    return {"conf": conf, "precision": p, "recall": r, "F1": 2 * p * r / max(p
+ r, 1e-9), "개수 틀린 사진": wrong}

# 1) valid 셋에서 최적 신뢰도 찾기
val_recs = collect(best_model, "valid")
val_sweep = pd.DataFrame([score(val_recs, c) for c in np.round(np.arange(0.05,
0.95, 0.05), 2)])
best_conf = float(val_sweep.loc[val_sweep["F1"].idxmax(), "conf"])
print(f"valid 셋 기준 최적 신뢰도: {best_conf} (F1 {val_sweep['F1'].max():.3
f})")

plt.figure(figsize=(7, 3.5))
for col in ["precision", "recall", "F1"]:
    plt.plot(val_sweep.conf, val_sweep[col], "o-", ms=3, label=col)
plt.axvline(0.25, color="gray", ls="--", label="default 0.25"); plt.axvline(bes
t_conf, color="red", ls="--", label=f"best {best_conf}")
plt.xlabel("conf threshold"); plt.ylim(0.5, 1.02); plt.grid(alpha=.3); plt.legen
d(); plt.title("valid set")

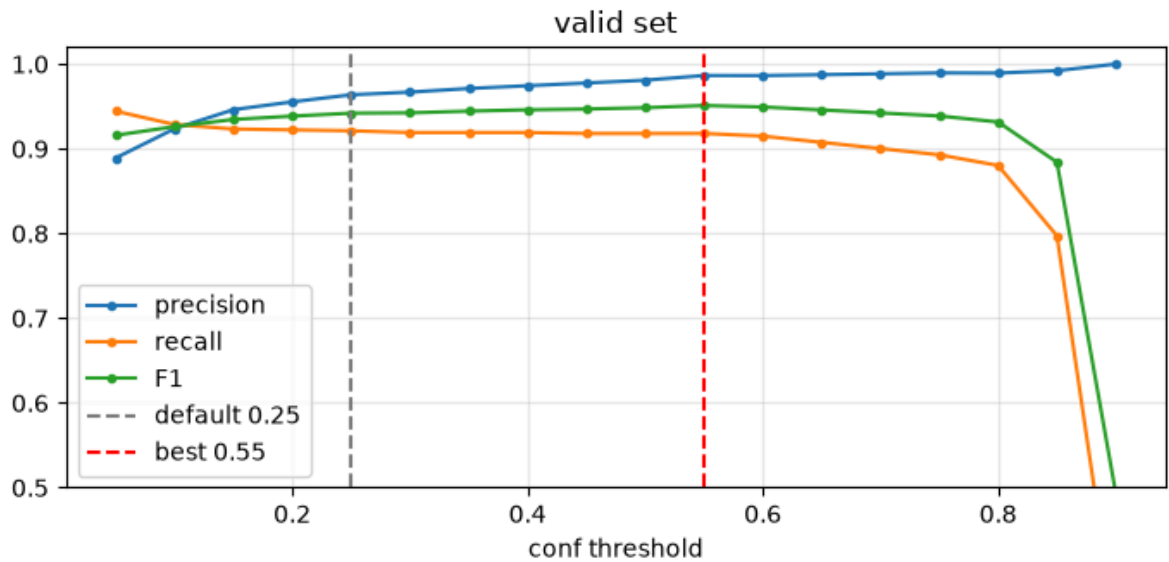
```

```
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_conf_opt.png", dpi=150); plt.show()

# 2) test 셋에서 기본값과 비교
test_recs = collect(best_model, "test")
conf_cmp = pd.DataFrame([score(test_recs, 0.25), score(test_recs, best_conf)], index=["기본값", "최적값"]).round(3)
conf_cmp
```

가장 좋은 모델: 11n\_e20

valid 셋 기준 최적 신뢰도: 0.55 (F1 0.951)



Out[63]:

|     | conf | precision | recall | F1    | 개수 | 틀린 사진 |
|-----|------|-----------|--------|-------|----|-------|
| 기본값 | 0.25 | 0.961     | 0.946  | 0.954 | 46 |       |
| 최적값 | 0.55 | 0.980     | 0.935  | 0.957 | 46 |       |

## 실험 2-2 결과 해석

### 결과

|                  | 신뢰도         | 정밀도          | 재현율   | F1           | 개수 틀린 사진 |
|------------------|-------------|--------------|-------|--------------|----------|
| 기본값              | 0.25        | 0.961        | 0.946 | 0.954        | 46 / 193 |
| 최적값 (valid에서 선택) | <b>0.55</b> | <b>0.980</b> | 0.935 | <b>0.957</b> | 46 / 193 |

### 분석 — H4는 방향은 맞지만 효과가 매우 작다

- valid 셋에서 F1이 가장 높은 신뢰도는 **0.55**였다. test 셋에 적용하니 정밀도는 0.961 → 0.980으로 올랐고, 재현율은 0.946 → 0.935로 조금 내렸다. **F1은 0.003 오르는 데 그쳤다.**
- 개수가 틀린 사진은 **46장 그대로**다. 사진별로 보면 0.55에서 **3장이 고쳐지고**(도장 중 북 박스가 걸러짐) **3장이 새로 틀렸다**(맞던 도장·신발이 빠짐).

### 효과가 작은 이유

- 걸러낼 헛것이 원래 적다.** 신뢰도 0.25 기준 헛것(FP)은 24개로, 맞힌 박스 약 600개에 비해 적다. 절반을 걸러도 전체 점수는 조금만 바뀐다.
- 헛것과 맞힌 박스의 신뢰도가 겹친다.** 맞힌 박스는 81%가 0.85 이상이지만 5%는 0.5 아래다. 헛것은 중앙값 0.53으로 0.5 근처에 퍼져 있다. 그래서 기준을 올리면 헛것과 함께 맞던 박스도 빠진다(고친 3장 = 새로 틀린 3장).
- 남은 오류의 대부분은 "신발 1개 놓침"(46장 중 29장)이다.** 이 신발들(가장자리·손에 든 신발)은 박스가 아예 없는 게 아니라 **신뢰도가 0.25보다 낮게** 나온다. 모델 자신이 확신하지 못하는 것이다. valid 곡선에서 보듯 기준을 0.55보다 낮출수록 F1은 오히려 떨어진다. 놓친 신발을 살리려고 기준을 낮추면 헛것이 더 늘기 때문에, 신뢰도 조절로는 풀 수 없다.

→ **결론:** 신뢰도 조절은 오탐을 줄이는 **운영 조정**으로는 쓸 만하지만(정밀도 0.98), 성능을 크게 올리는 방법은 아니다. 기본값 0.25로도 거의 최적이다. 2-1과 같은 결론으로, 남은 오류를 줄이려면 모델이 확신하지 못하는 장면(가장자리·손에 든 신발)의 **데이터를 보강**해야 한다.

## 2-3. 추가 샘플 데이터 + 촬영 조건 변화

```
In [64]: def _dark(im): return cv2.convertScaleAbs(im, alpha=0.45, beta=0) # 조명 부족
def _bright(im): return cv2.convertScaleAbs(im, alpha=1.3, beta=60) # 과노출
def _blur(im): return cv2.GaussianBlur(im, (9, 9), 0) # 초점 흐림 / 움직임
def _noise(im):
    n = np.random.default_rng(0).normal(0, 25, im.shape) # 센서 노이즈
    return np.clip(im.astype(np.float32) + n, 0, 255).astype(np.uint8)

CONDS = {"original": None, "dark": _dark, "bright": _bright, "blur": _blur, "noise": _noise}
rows, examples = [], {}
ex_img = STAMP / "test/images/clip1_mp4-33_jpg.rf.95be2fb4fbae44fb12fd600d1d966f76.jpg"
for cond, fn in CONDS.items():
    d = ROOT / "datasets" / "stamp_robust" / cond
    if not (d / "data.yaml").exists():
        (d / "test" / "images").mkdir(parents=True, exist_ok=True)
        shutil.copypath(STAMP / "test" / "labels", d / "test" / "labels", dirs_exist_ok=True)
        for p in (STAMP / "test" / "images").glob("*"):
            im = cv2.imread(str(p)); cv2.imwrite(str(d / "test" / "images" / p.name), fn(im) if fn else im)
        (d / "data.yaml").write_text(yaml.safe_dump({"path": d.as_posix(), "train": "test/images", "val": "test/images", "test": "test/images", "names": cfg["names"]}), encoding="utf-8")
        v = best_model.val(data=str(d / "data.yaml"), split="test", device=DEVICE, verbose=False, plots=False,
                           project=str(RUNS), name="robust", exist_ok=True)
        rows.append({"condition": cond, "mAP50": v.box.map50, "mAP50-95": v.box.map,
                     "shoes R": v.box.class_result(0)[1], "stamp R": v.box.class_result(1)[1]})
    examples[cond] = best_model.predict(str(d / "test" / "images" / ex_img.name), device=DEVICE, verbose=False)[0].plot()[ :, :, :-1]
robust = pd.DataFrame(rows).round(3)
robust.to_csv(ROOT / "results" / "stamp_robustness.csv", index=False)

fig, axes = plt.subplots(1, 5, figsize=(22, 4.5))
for ax, (cond, im) in zip(axes, examples.items()):
    ax.imshow(im); ax.set_title(cond); ax.axis("off")
plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_robust_examples.png", dpi=110); plt.show()
robust
```

Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)  
 YOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.4 GFLOPs  
 WARNING val: Slow image access detected (ping: 0.20.0 ms, read: 4.30.5 MB/s, size: 49.6 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>  
 val: New cache created: C:\Users\LQ5060\Desktop\CV-YOLO\datasets\stamp\_robust\original\test\labels.cache

all 193 633 0.977 0.93 0.983

0.634

Speed: 1.8ms preprocess, 6.8ms inference, 0.0ms loss, 1.4ms postprocess per image  
 Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

YOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.4 GFLOPs  
 WARNING val: Slow image access detected (ping: 0.20.1 ms, read: 4.00.4 MB/s, size: 38.4 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

... (11줄 생략) ...

WARNING val: Slow image access detected (ping: 0.20.1 ms, read: 3.50.8 MB/s, size: 37.2 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

val: New cache created: C:\Users\LQ5060\Desktop\CV-YOLO\datasets\stamp\_robust\blur\test\labels.cache

all 193 633 0.958 0.892 0.939

0.483

Speed: 1.9ms preprocess, 7.6ms inference, 0.0ms loss, 1.3ms postprocess per image  
 Ultralytics 8.4.154 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 5060 Laptop GPU, 8151MiB)

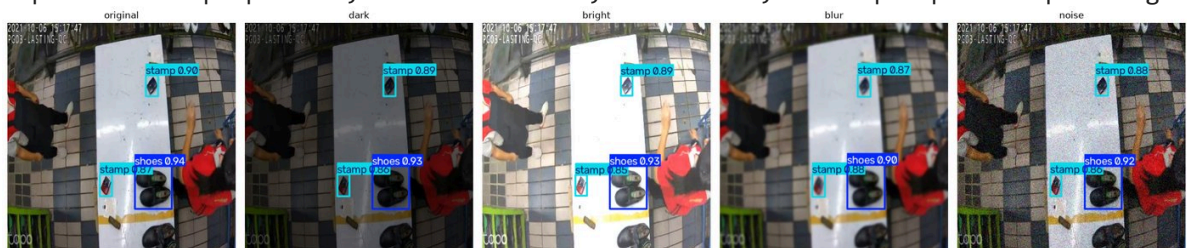
YOLO11n summary (fused): 100 layers, 2,582,542 parameters, 0 gradients, 6.4 GFLOPs  
 WARNING val: Slow image access detected (ping: 0.20.1 ms, read: 13.71.2 MB/s, size: 139.1 KB). Use local storage instead of remote/mounted storage for better performance. See <https://docs.ultralytics.com/guides/model-training-tips>

val: New cache created: C:\Users\LQ5060\Desktop\CV-YOLO\datasets\stamp\_robust\noise\test\labels.cache

all 193 633 0.955 0.819 0.909

0.589

Speed: 1.6ms preprocess, 3.7ms inference, 0.0ms loss, 1.2ms postprocess per image



Out[64]:

|   | condition | mAP50 | mAP50-95 | shoes R | stamp R |
|---|-----------|-------|----------|---------|---------|
| 0 | original  | 0.983 | 0.634    | 0.869   | 0.990   |
| 1 | dark      | 0.981 | 0.627    | 0.852   | 0.984   |
| 2 | bright    | 0.972 | 0.635    | 0.816   | 0.988   |
| 3 | blur      | 0.939 | 0.483    | 0.811   | 0.974   |
| 4 | noise     | 0.909 | 0.589    | 0.677   | 0.961   |

### 실험 2-3 결과 해석

| 조건             | mAP50        | mAP50-95     | shoes Recall | stamp Recall |
|----------------|--------------|--------------|--------------|--------------|
| original       | 0.983        | 0.634        | 0.869        | 0.990        |
| dark (밝기 45%)  | 0.981        | 0.627        | 0.852        | 0.984        |
| bright (과노출)   | 0.972        | 0.635        | 0.816        | 0.988        |
| blur (흐림)      | <b>0.939</b> | <b>0.483</b> | 0.811        | 0.974        |
| noise (센서 노이즈) | <b>0.909</b> | 0.589        | <b>0.677</b> | 0.961        |

- **조명 변화(dark/bright)**에는 mAP 변화가 거의 없다. 학습 때 Ultralytics가 기본으로 밝기·채도를 흔드는 증강( `hsv_v=0.4` 등)을 적용했기 때문으로 보인다. 다만 과노출에서는 신발 Recall이 0.87에서 0.82로 줄었다.
- **흐림**은 mAP50-95가 0.634에서 0.483으로 **가장 크게 떨어졌다**. 대충은 찾지만 경계가 뭉개져 박스가 덜 정확해진다.
- **노이즈**에서는 **신발 Recall이 0.87에서 0.68로 급락했다**. 반면 도장은 모든 조건에서 Recall 0.96 이상을 유지했다. 어두운 신발의 무늬가 노이즈에 묻히는 반면, 도장은 흰 작업대 위에서 색 대비가 커서 버티는 것으로 보인다.
- 예시 사진(한 장)에서는 다섯 조건 모두 같은 박스(도장 2, 신발 1)를 찾았다. 아래쪽 가장자리의 잘린 신발은 모든 조건에서 놓쳤다.

→ 현장 적용 시에는 **카메라 초점 고정(흐림 방지)과 충분한 조명(노이즈 감소)** 이 가장 중요하다. 학습 데이터에 **블러·노이즈 증강**을 추가하면 개선될 것으로 기대한다.

## 2-4. 틀린 사례 모아보기

```
In [65]: # test 셋에서 틀린 사례 찾기: 정답 개수와 예측 개수가 다른 이미지
def count_gt(p):
    lines = [l for l in (STAMP / "test" / "labels" / (p.stem + ".txt")).read_text().splitlines() if l.strip()]
    c = [int(l.split()[0]) for l in lines]
    return c.count(0), c.count(1)

wrong = []
for r in best_model.predict([str(p) for p in sorted((STAMP / "test" / "images").glob("*"))], conf=0.25,
                             device=DEVICE, stream=True, verbose=False):
    p = Path(r.path); gs, gt = count_gt(p)
    pc = r.bboxes.cls.tolist(); ps, pt = pc.count(0), pc.count(1)
    if (gs, gt) != (ps, pt):
        wrong.append({"image": p.name, "gt shoes": gs, "pred shoes": ps, "gt stamp": gt, "pred stamp": pt, "_plot": r.plot()[0, :, :-1]})
print(f"개수가 틀린 이미지: {len(wrong)} / 193")
wrong_df = pd.DataFrame(wrong).drop(columns="_plot")
display(wrong_df.head(10))
if wrong:
    k = min(6, len(wrong))
    fig, axes = plt.subplots(1, k, figsize=(4 * k, 4), squeeze=False)
    for ax, w in zip(axes[0], wrong[:k]):
        ax.imshow(w["_plot"]); ax.set_title(f"GT s{w['gt shoes']}/t{w['gt stamp']} pred s{w['pred shoes']}/t{w['pred stamp']}", fontsize=9); ax.axis("off")
    plt.tight_layout(); plt.savefig(ROOT / "report" / "images" / "stamp_errors.png", dpi=110); plt.show()
```

개수가 틀린 이미지: 46 / 193

|   | image                                             | gt shoes | pred shoes | gt stamp | pred stamp |
|---|---------------------------------------------------|----------|------------|----------|------------|
| 0 | clip1_mp4-1007_jpg.rf.797cad76f7531b9a2ea20751... | 3        | 2          | 1        | 1          |
| 1 | clip1_mp4-1018_jpg.rf.5c9c9fc7fa337f9d2a814dbf... | 3        | 2          | 2        | 2          |
| 2 | clip1_mp4-1075_jpg.rf.57825344122889ed12f7a9d0... | 4        | 4          | 1        | 3          |
| 3 | clip1_mp4-1076_jpg.rf.eb74636993a043016bd5feee... | 4        | 4          | 2        | 3          |
| 4 | clip1_mp4-1092_jpg.rf.a4d3481c456a22d664f8df3c... | 3        | 3          | 2        | 5          |
| 5 | clip1_mp4-118_jpg.rf.2a50eb1e3bca5710179bba17f... | 4        | 3          | 2        | 2          |
| 6 | clip1_mp4-149_jpg.rf.4463b36f6d52e4b5a91b9159f... | 2        | 1          | 2        | 2          |
| 7 | clip1_mp4-157_jpg.rf.a09074ebd2a276b0ae3a852d6... | 2        | 1          | 2        | 4          |
| 8 | clip1_mp4-227_jpg.rf.baa51515a208848a50a6e30ab... | 1        | 0          | 2        | 2          |
| 9 | clip1_mp4-233_jpg.rf.790d2884ae480e950f888ad30... | 2        | 1          | 2        | 2          |



## 실험 2-4 결과 해석

conf 0.25 기준으로 **193장 중 46장(24%)** 에서 정답 개수와 예측 개수가 달랐다. 틀린 사진을 직접 보면 세 가지 유형이 보인다.

1. **이미지 가장자리에 잘린 신발을 놓침** (정답 신발 3 → 예측 2): 아래쪽 끝에 일부만 보이는 신발이 라벨에는 있지만 모델은 찾지 못했다. 실험 2-3 예시에서도 같은 신발을 모든 조건에서 놓쳤다.
2. **작업자가 들고 있거나 손에 가려진 신발을 놓침** (정답 신발 4 → 예측 3): 작업대 위의 신발과 모양·각도가 달라서 못 찾는다. 혼동행렬에서 본 "shoes 12% 놓침"의 주 원인이다.
3. **도장 박스가 더 많이 나옴** (정답 도장 2 → 예측 3~5): 작업자 손 근처에 도장 박스가 겹쳐서 여러 개 생기거나(0.66 등), 들고 있는 신발 위에 낮은 신뢰도(0.39)의 도장 박스가 추가로 생겼다.

→ 개선 방향: 가장자리에 걸친 신발과 손에 든 신발 사진을 더 모아 학습하고, 도장은 임계값을 0.5 이상으로 올리거나 NMS IoU를 낮추면 중복 박스가 줄어들 것이다.

# 목표 기준 평가 및 성과

| 목표             | 수행 내용                                                          | 결과                                                                                              |
|----------------|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| ① 추론 결과 구조 이해  | Results / Boxes 구조, xywh·xyxy·정규화 좌표를 직접 계산해 검증, 클래스 필터링, crop | 직접 계산한 값과 xywhn / xyxyn 이 일치함을 확인                                                               |
| ② 제조 데이터 학습·평가 | stamp v10으로 YOLOv8n 20 에폭 파인튜닝 (과제 코드 그대로)                     | val mAP@.5 0.985 / mAP@.5:.95 0.640 (과제 예시 0.982 / 0.656과 같은 수준)                                |
| ③ 성능 범위 실험     | 임계값·NMS·추가 샘플·모델 3종 (YOLO11n 포함)·에폭·촬영 조건 5종·오류 사례 분석          | 모델·에폭보다 데이터(작은 도장, 가장자리·손에 가려진 신발)와 촬영 환경(흐림·노이즈) 이 성능을 결정함                                     |
| (추가) 레포 리팩터링   | 노트북 흐름을 src/ · tools/ 스크립트로 분리, README 작성, 제출용 PDF 자동 생성       | <a href="https://github.com/hwang-ye-song/CV_YOLO">https://github.com/hwang-ye-song/CV_YOLO</a> |

## 성과 요약

- 20 에폭(약 3.5분)의 가벼운 학습만으로 검증 셋의 도장 **Recall 1.000**. "도장이 찍혔는지"를 확인하는 공정 목적에는 충분한 수준이다.
- 다른 모델(YOLO11n)을 적용해 더 작은 모델로 같거나 조금 나은 성능(test mAP50-95 0.634)을 얻었다.
- 약점은 **가장자리·손에 가려진 신발 미탐**과 **흐림·노이즈 환경에서의 성능 저하**이다. 원인과 개선 방향(데이터 추가, 증강, 카메라·조명 고정, 클래스별 임계값)을 실험으로 찾아냈다.

# 개인 회고

## 진행하면서 막혔던 부분과 해결

| 막힌 부분                                                       | 원인                                                                             | 해결                                                                                       |
|-------------------------------------------------------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Roboflow 링크로 받은 데이터가 이미지 3장뿐                                | 공유 링크가 다른 버전(v2, s, t, amp) 1개 클래스)이었다                                         | 데이터셋 버전 8개의 분할 수를 비교해, 과제와 같은 <b>v10</b> (train 1,100 / valid 273 / test 193)을 찾아 다시 받았다 |
| 명령줄로 받은 zip이 HTML 파일                                        | <code>universe.roboflow.com</code> 링크가 보안 확인 페이지를 돌려줬다                         | Roboflow 안내 형식인 <code>app.roboflow.com/ds/...</code> 주소로 받았다                             |
| RTX 5060에 맞는 PyTorch 필요                                     | RTX 50 시리즈는 CUDA 12.8 빌드가 필요하다                                                 | <code>--index-url .../cu128</code> 로 PyTorch를 설치했다                                       |
| 과제 코드 <code>import requests=</code> 오류                      | 문법 오류(오타)                                                                      | <code>import requests</code> 로 고쳤다                                                       |
| 과제의 <code>./runs/detect/predict/bus.jpg</code> 를 못 찾음       | 결과가 홈 폴더( <code>C:\Users\...\runs</code> )에 저장되고 있었다                           | 이 프로젝트 커널에만 YOLO 설정 폴더를 따로 지정해, 결과가 노트북 폴더의 <code>runs/</code> 에 저장되게 했다                 |
| <code>data.yaml</code> 의 <code>../train/images</code> 상대 경로 | 실행 위치에 따라 경로가 달라진다                                                             | 데이터셋을 과제와 같은 <code>../datasets/stamp</code> 위치에 두었다. 개인 실험용으로는 절대경로 yaml을 따로 만들었다        |
| PDF에 숫자 수천 개가 수십 쪽 찍힘                                       | <code>model.train()</code> 반환값과 <code>print(metrics)</code> 가 곡선 데이터까지 모두 출력했다 | 노트북은 과제 코드 그대로 두고, PDF로 바꿀 때 긴 출력은 앞뒤만 남기게 했다                                            |
| 다시 학습하니 결과 숫자가 달라짐                                          | GPU 학습은 시드를 고정해도 실행마다 조금씩 달라진다 (mAP50-95 $\pm 0.01$ )                          | 해석을 최종 실행 결과에 맞춰 다시 썼고, 모델 비교는 이 흔들림 폭을 고려해 판단했다                                         |
| 노트북 결과가 덮어써질 뻔함                                             | 실행 전 버전이 열린 JupyterLab 탭이 자동 저장했다                                              | 백그라운드 실행 중에는 Jupyter 서버를 종료해 같은 노트북이 열려 있지 않게 했다                                         |

## KPT

### • Keep

- 기본 코드를 그대로 따라가며 좌표 값을 **직접 계산해 확인**한 것. 라이브러리 출력의 의미를 확실히 이해할 수 있었다.
- 모델 하나의 점수에서 멈추지 않고 **왜 그 점수가 나오는지**(작은 물체, 가려짐, 흐림·노이즈)까지 실험으로 확인한 것.
- 틀린 사진을 직접 모아 본 것. 숫자만으로는 알 수 없던 "가장자리 신발" 문제를 찾았다.

### • Problem

- 데이터셋 **버전과 분할 수를 먼저 확인하지 않아** 시간을 썼다.

- 실행 환경(결과 저장 위치, 상대경로)이 과제 환경과 달라 과제 코드가 그대로 돌지 않았다.
- 한 번의 학습 결과로 모델을 비교하면, 실행마다 생기는 흔들림과 구분하기 어렵다.
- Try
  - 학습 데이터에 **블러·노이즈 증강**을 넣고 촬영 조건 실험을 다시 해서 개선 폭을 확인한다.
  - 모델 비교는 **시드를 바꿔 여러 번 학습해 평균**을 낸다.
  - 클래스별로 다른 신뢰도 임계값을 적용하는 후처리를 만든다.
  - 직접 촬영한 제품 사진으로 **정상/불량 영역 탐지**를 학습해 실제 현장 적용 가능성을 검증한다 (다음 프로젝트).

## AAR (After Action Review)

| 질문          | 답                                                                          |
|-------------|----------------------------------------------------------------------------|
| 원래 의도한 것은?  | 과제 코드를 재현하고, 모델을 바꿔 성능을 끌어올린다                                              |
| 실제로 일어난 일은? | 재현은 성공했지만(mAP50 0.985), 모델·에폭을 바꿔도 성능 차이가 실행마다 생기는 흔들림( $\pm 0.01$ ) 수준이었다 |
| 차이가 생긴 이유는? | 성능 한계가 모델 용량이 아니라 <b>데이터 특성</b> (작은 도장의 박스 정밀도, 가장자리·손에 가려진 신발)에 있었다       |
| 다음에 할 것은?   | 모델 크기보다 <b>데이터 보강·증강과 촬영 환경 개선</b> 에 먼저 시간을 쓴다. 성능이 멈추면 틀린 사례부터 본다         |

## 동료 피드백 (선택)

- (받은 피드백을 여기에 기록)